A Survey of Category Theory and Deep Learning

Bridging Abstract Mathematics and Modern Artificial Intelligence

Serge Youmbi

PhD Researcher

University of Derby, United Kingdom

A Comprehensive Literature Review

For Deep Learning Practitioners, AI Researchers, and Machine Learning Engineers

15th December 2025

Abstract

This survey presents a comprehensive examination of the emerging intersection between category theory—the abstract mathematical study of structure and composition—and deep learning, the driving force behind contemporary artificial intelligence. We trace the historical development of categorical approaches to machine learning, from early type-theoretic foundations through to modern frameworks for understanding neural network composition, automatic differentiation, and learned representations. The survey is organised around six thematic clusters: compositional foundations, differentiation and learning dynamics, geometric and equivariant structures, semantic and logical frameworks, probabilistic and information-theoretic perspectives, and implementation paradigms. Throughout, we emphasise the bidirectional nature of this relationship: category theory provides deep learning with rigorous compositional semantics and new architectural principles, whilst deep learning challenges category theory with novel mathematical structures demanding formalisation. We critically assess the maturity, tractability, and

empirical validation of existing approaches, identify significant gaps in the literature, and outline promising directions for future research. This survey is intended to be accessible to practitioners with machine learning backgrounds whilst providing sufficient mathematical depth to serve as a reference for researchers working at this interdisciplinary frontier.

Keywords: Category Theory, Deep Learning, Neural Networks, Functors, Monoidal Categories, Automatic Differentiation, Compositional Semantics, Geometric Deep Learning

Contents

1 Introduction **7**
1.1 Motivation: Why Category Theory for Deep Learning? 7
1.2 Scope and Organisation 8
1.3 Historical Context 9

2 Categorical Foundations for Deep Learning **10**
2.1 Categories: Objects and Morphisms 11
2.2 Functors: Structure-Preserving Maps 12
2.3 Natural Transformations: Morphisms Between Functors 13
2.4 Monoidal Categories and String Diagrams 14
2.5 Adjunctions and Universal Properties 16
2.6 The Yoneda Lemma and Representability 16

3 Compositional Foundations of Deep Learning **17**
3.1 Neural Networks as Morphisms 18
3.2 The Para Construction 18
3.3 String Diagrams for Neural Architectures 19
3.4 Polynomial Functors for Architecture Specification 20
3.5 Operads and Wiring Diagrams 21
3.6 Symmetric Monoidal Categories and the Interchange Law 22
3.7 Compact Closed Categories and Duality 22
3.8 Traced Monoidal Categories and Feedback 23

4 Categorical Perspectives on Differentiation and Learning **24**
4.1 Cartesian Differential Categories 24
4.2 Reverse Derivative Categories and Backpropagation 25
4.3 Lenses and Optics for Bidirectional Data Flow 26
4.4 Gradient Descent as a Functor 27
4.5 The Seven Axioms of Cartesian Differential Categories 27
4.6 Automatic Differentiation Modes 28
4.7 Loss Landscapes and Categorical Topology 29

5 Geometry, Symmetry, and Equivariant Neural Networks **29**
5.1 Groups, Representations, and Functor Categories 30
5.2 Convolutional Networks as Functorial Constructions 31
5.3 Graph Neural Networks and Sheaf Theory 32
5.4 The Yoneda Perspective on Representations 32

5.5 Gauge Equivariance and Fibre Bundles . . . 33
5.6 Kan Extensions and Universal Constructions in Representation Learning 33
5.7 Limits, Colimits, and Architectural Design . . . 34

6 Semantic and Logical Frameworks 35
6.1 Categorical Compositional Distributional Semantics . . . 36
6.2 Type Systems for Neural Networks . . . 37
6.3 Topos-Theoretic Approaches . . . 37
6.4 Linear Logic and Resource Sensitivity . . . 38
6.5 Categorical Models of Attention and Transformers . . . 39
6.6 Functorial Semantics for Neural Language Models . . . 39

7 Probabilistic and Information-Theoretic Perspectives 41
7.1 Markov Categories and Probabilistic Programming . . . 41
7.2 The Kleisli Category for Probability . . . 41
7.3 Bayesian Deep Learning . . . 42
7.4 Information Geometry Through Categorical Lenses . . . 43
7.5 Coalgebraic Perspectives on Recurrence . . . 44
7.6 Synthetic Probability and Conditional Independence . . . 44
7.7 Categorical Entropy and Information Functors . . . 45

8 Implementation and Programming Paradigms 46
8.1 Functional Programming and Deep Learning . . . 46
8.2 Differentiable Programming Languages . . . 47
8.3 Recursion Schemes for Neural Architectures . . . 48
8.4 Categorical DSLs for Deep Learning . . . 48
8.5 Compiling to Categories . . . 49
8.6 Effect Systems and Differentiable Programming . . . 50
8.7 Production Systems and Scalability . . . 50

9 Case Studies: Categorical Deep Learning in Practice 51
9.1 Case Study 1: Equivariant Neural Networks for Molecular Property Prediction . . . 51
9.1.1 The Categorical Framework . . . 51
9.1.2 Architectural Implications . . . 52
9.1.3 Results and Insights . . . 52
9.2 Case Study 2: DisCoCat for Sentiment Analysis . . . 53
9.2.1 The Compositional Model . . . 53
9.2.2 Results and Limitations . . . 54

9.3 Case Study 3: Categorical Automatic Differentiation in JAX 54
9.3.1 Functional Transformations as Functors 54
9.3.2 Categorical Composition in Practice 55
9.3.3 Impact 55
9.4 Case Study 4: Sheaf Neural Networks for Heterophilous Graphs 55
9.4.1 The Sheaf Perspective 56
9.4.2 Results 56

10 Gaps, Challenges, and Future Directions 56
10.1 Theoretical Gaps 57
10.2 Application Gaps 57
10.3 Methodological Gaps 58
10.4 Promising Synthesis Opportunities 59
10.5 Detailed Analysis of Key Open Problems 59
10.5.1 The Expressivity Problem 59
10.5.2 The Generalisation Puzzle 60
10.5.3 Compositional Semantics for Deep Learning 61
10.6 Interdisciplinary Bridges 61
10.7 A Research Agenda for Categorical Deep Learning 62
10.7.1 Short-Term Goals 62
10.7.2 Medium-Term Goals 63
10.7.3 Long-Term Goals 63

11 Conclusion 63
11.1 Key Insights 64
11.2 The State of the Field 64
11.3 Future Trajectory 65
11.4 A Vision for Categorical Deep Learning 65
11.5 Reflections on the Categorical Method 66
11.6 Caveats and Limitations 66

References 68

A Mathematical Background and Notation 76
A.1 Set-Theoretic Foundations 76
A.2 The Category of Sets 76
A.3 The Category of Vector Spaces 77
A.4 Smooth Manifolds and Euclidean Spaces 77
A.5 Notation Summary 77

A.6 Diagram Conventions 77

B Glossary of Categorical Terms **78**

C Supplementary Diagrams **79**

D Further Reading **81**

D.1 Category Theory Foundations 81
D.2 Categorical Logic and Type Theory 81
D.3 Monoidal Categories and String Diagrams 81
D.4 Categorical Probability 81
D.5 Deep Learning Foundations 82
D.6 Categorical Deep Learning 82

1 Introduction

The remarkable success of deep learning over the past decade has transformed artificial intelligence from a specialised research discipline into a pervasive technology reshaping society.[1] Neural networks now achieve superhuman performance on tasks ranging from image recognition (He et al., 2016) to strategic game playing (Silver et al., 2017), and have become essential tools across scientific disciplines. Yet this empirical success has far outpaced our theoretical understanding: we lack rigorous frameworks explaining *why* deep learning works, *what* computations neural networks perform, and *how* their components compose to yield emergent capabilities.

Category theory, developed in the mid-twentieth century as a unifying language for mathematics, offers a promising foundation for addressing these questions.[2] At its core, category theory is the mathematics of *composition*—it studies how mathematical objects relate through structure-preserving maps and how these maps combine. This compositional perspective aligns naturally with the architecture of neural networks, which are fundamentally compositions of parameterised functions assembled into hierarchical structures.

1.1 Motivation: Why Category Theory for Deep Learning?

The application of category theory to deep learning is motivated by several convergent factors, each addressing fundamental limitations in current theoretical frameworks. First, neural network architectures exhibit rich compositional structure that existing formalisms struggle to capture. A typical deep network comprises convolutions, normalisations, nonlinearities, attention mechanisms, and residual connections, composed in intricate patterns. Category theory provides a vocabulary—functors, natural transformations, monoidal products—precisely suited to describing such compositional systems.[3]

Second, the training of neural networks through gradient-based optimisation has a natural categorical interpretation. Backpropagation, the workhorse algorithm of deep learning, computes derivatives by composing local gradients—a process that category

[1] The term "deep learning" refers to machine learning methods based on artificial neural networks with multiple layers, enabling hierarchical feature learning. See LeCun et al. (2015) for a comprehensive overview.

[2] Category theory was introduced by Eilenberg and Mac Lane (1945) to formalise relationships between algebraic topology and abstract algebra. For accessible introductions, see Awodey (2010) and Leinster (2014).

[3] This compositional perspective has proven extraordinarily fruitful in quantum computing, where categorical methods revolutionised our understanding of quantum protocols (Coecke and Kissinger, 2017).

theory captures through *reverse derivative categories* (Cockett and Cruttwell, 2020) and the *lens* formalism from functional programming (Riley, 2018). These categorical abstractions unify automatic differentiation with broader patterns of bidirectional information flow.

Third, modern deep learning increasingly emphasises *equivariance*—designing networks that respect symmetries in their input domains. Equivariant neural networks for 3D point clouds (Fuchs et al., 2020), molecular graphs (Satorras et al., 2021), and physical systems (Brandstetter et al., 2022) all exploit group-theoretic structure. Category theory, through its treatment of group actions as functors, provides the natural framework for understanding and extending these architectures.[4]

1.2 Scope and Organisation

This survey comprehensively examines the intersection of category theory and deep learning, organised around six thematic clusters reflecting the primary research directions in this emerging field.

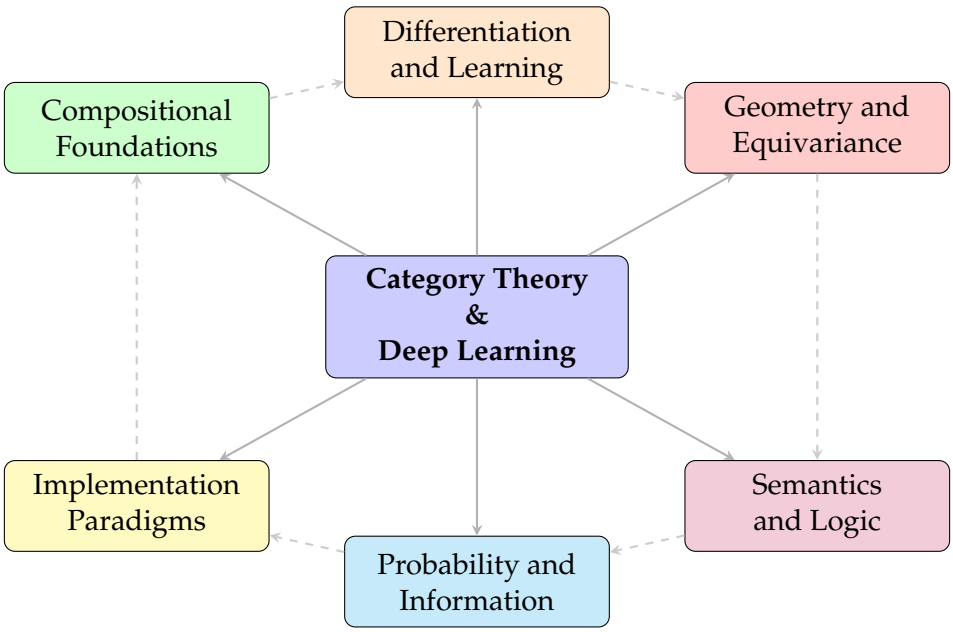


Figure 1: Organisation of the survey around six thematic clusters. Solid arrows indicate primary relationships with the central topic; dashed lines show inter-cluster connections that will be developed throughout the survey.

Section 2 introduces the essential category-theoretic concepts required for the remainder of the survey. We present categories, functors, and natural transformations

[4]The geometric deep learning programme of Bronstein et al. (2021) explicitly acknowledges these categorical underpinnings.

with an emphasis on intuition and examples drawn from machine learning contexts. This section serves as both a primer for readers new to category theory and a reference establishing our notational conventions.

Section 3 examines *compositional foundations*—how categorical structures formalise neural network composition. We discuss monoidal categories and string diagrams, the Para construction for parameterised morphisms, polynomial functors for architecture specification, and operadic approaches to network assembly.

Section 4 addresses *differentiation and learning*, exploring categorical treatments of automatic differentiation, gradient-based optimisation, and the backward pass. Central topics include Cartesian differential categories, reverse derivative categories, and the lens/optic formalism.

Section 5 surveys *geometric and equivariant* approaches, examining how category theory illuminates geometric deep learning, group equivariance, and the role of symmetry in neural architectures. We discuss functor categories, sheaf-theoretic perspectives, and connections to the Yoneda lemma.

Section 6 explores *semantic and logical* frameworks, including categorical compositional distributional semantics (DisCoCat), type-theoretic foundations, and topos-theoretic approaches to neural network meaning.

Section 7 examines *probabilistic and information-theoretic* perspectives, covering Markov categories, synthetic probability theory, and categorical treatments of Bayesian deep learning.

Section 8 discusses *implementation paradigms*, reviewing how categorical concepts manifest in functional programming, differentiable programming languages, and production deep learning systems.

Section 10 identifies gaps in the literature and outlines promising directions for future research, whilst **Section 11** offers concluding reflections on the trajectory of this interdisciplinary field.

1.3 Historical Context

The application of categorical methods to computation has a distinguished history predating deep learning by decades. The Curry-Howard-Lambek correspondence, established through the work of Curry and Feys (1958), Howard (1980), and Lambek (1980), revealed profound connections between logic, type theory, and category theory.[5] This correspondence underlies the categorical semantics of programming

[5]This correspondence shows that propositions correspond to types, proofs to programs, and the rules of inference to categorical constructions. It remains foundational to programming language theory.

languages and laid groundwork for understanding computation through categorical lenses.

The specific application of category theory to neural networks, however, is more recent. Early intimations appeared in the 1990s with Eilenberg and Mac Lane's observation that neural network composition resembled categorical composition, but systematic development awaited the twenty-first century. The landmark paper by Fong et al. (2019) demonstrated that backpropagation could be understood through the lens formalism, igniting contemporary interest.[6] Shortly thereafter, Cruttwell et al. (2022) provided a comprehensive categorical framework for gradient-based learning, establishing the *Para* construction as a central organising principle.

Parallel developments in applied category theory—spearheaded by the Topos Institute and researchers including David Spivak, Brendan Fong, and their collaborators—created an intellectual ecosystem receptive to categorical approaches in applied domains.[7] The annual Applied Category Theory conference, established in 2018, has become a venue for presenting categorical perspectives on machine learning.

Today, the field stands at an inflection point. Sufficient theoretical infrastructure exists to support serious categorical investigation of deep learning, yet many fundamental questions remain open. This survey aims to consolidate existing knowledge, identify patterns across disparate research threads, and chart paths forward for this promising interdisciplinary endeavour.

The rapid growth of this research area is evidenced by the cumulative publication record. As illustrated in Figure 2, the number of papers addressing categorical approaches to machine learning has accelerated markedly since 2018, reflecting both the maturation of applied category theory as a discipline and the deep learning community's increasing appetite for rigorous mathematical foundations. This trajectory suggests that categorical deep learning is transitioning from a niche theoretical pursuit to a recognised research programme with substantial momentum.[8]

2 Categorical Foundations for Deep Learning

Before examining specific applications, we establish the categorical vocabulary required throughout this survey. This section introduces the fundamental concepts—categories,

[6]This work built upon earlier insights from functional programming, where lenses had been developed as a pattern for bidirectional data access (Foster et al., 2007).

[7]The textbook *Seven Sketches in Compositionality* by Fong and Spivak (2019) exemplifies this accessible approach to applied category theory.

[8]The data comprises papers from arXiv (cs.LG, cs.AI, math.CT, stat.ML), major machine learning venues (NeurIPS, ICML, ICLR), and the Applied Category Theory conference proceedings, identified through systematic keyword searches as described in Section 1.

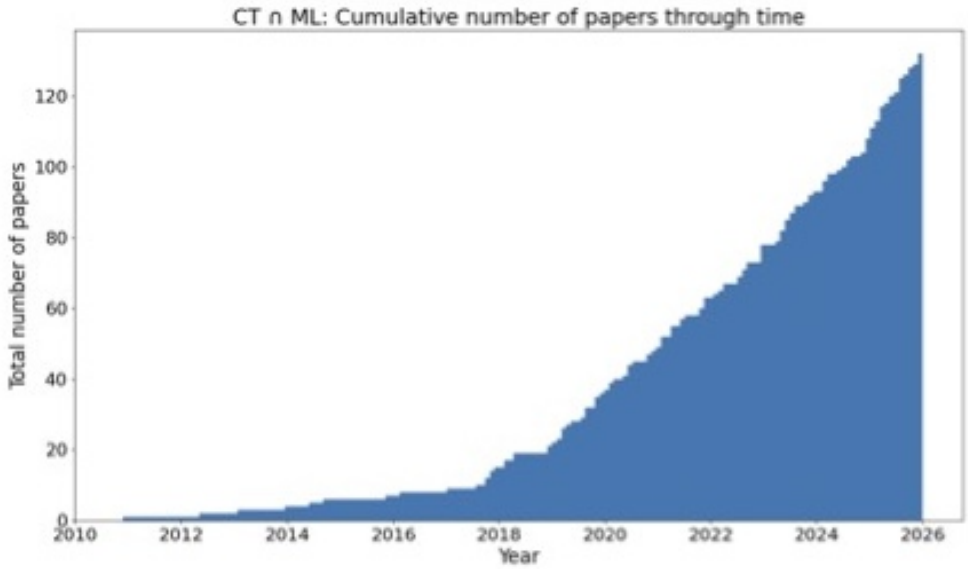


Figure 2: Cumulative number of research publications at the intersection of category theory and machine learning over time. The pronounced acceleration from 2018 onwards coincides with seminal works on categorical backpropagation (Fong et al., 2019) and the establishment of the Applied Category Theory conference series. This growth trajectory underscores the increasing recognition that categorical methods offer valuable perspectives on deep learning foundations.

functors, natural transformations, and related structures—with an emphasis on building intuition through examples relevant to machine learning.[9]

2.1 Categories: Objects and Morphisms

A *category* $\mathcal{C}$ consists of a collection of *objects* and, for each pair of objects A and B, a collection of *morphisms* (or *arrows*) from A to B, denoted $\mathrm{Hom}_{\mathcal{C}}(A, B)$ or simply $\mathcal{C}(A, B)$. Categories must satisfy two fundamental properties: the existence of identity morphisms and the associativity of composition.

Definition 2.1 (Category)**.** A *category* $\mathcal{C}$ comprises:

- A collection $\mathrm{ob}(\mathcal{C})$ of objects;
- For each pair of objects $A, B \in \mathrm{ob}(\mathcal{C})$, a collection $\mathcal{C}(A, B)$ of morphisms;
- For each object A, an identity morphism $\mathrm{id}_A \in \mathcal{C}(A, A)$;

[9]Readers already familiar with category theory may wish to skim this section for notational conventions before proceeding to Section 3. For comprehensive treatments, see Mac Lane (2013) or Riehl (2017).

- For each triple of objects A, B, C, a composition operation

$$\circ : \mathcal{C}(B, C) \times \mathcal{C}(A, B) \to \mathcal{C}(A, C)$$

subject to the axioms:

1. **Identity**: For all $f : A \to B$, we have $\mathrm{id}_B \circ f = f = f \circ \mathrm{id}_A$.
2. **Associativity**: For all $f : A \to B$, $g : B \to C$, $h : C \to D$, we have $h \circ (g \circ f) = (h \circ g) \circ f$.

The power of category theory lies in its generality: this definition encompasses an enormous range of mathematical structures. Several categories are particularly relevant to deep learning:

Example 2.1 (Categories Relevant to Deep Learning).

- **Set**: Objects are sets; morphisms are functions. This is the archetypal example.
- $\mathbf{Vect}_{\mathbb{R}}$: Objects are real vector spaces; morphisms are linear maps. Neural network layers often operate between vector spaces.
- **Smooth**: Objects are smooth manifolds; morphisms are smooth functions. This category is central to geometric approaches.
- **Euc**: Objects are Euclidean spaces $\mathbb{R}^n$; morphisms are smooth functions. Much of deep learning operates here.

The categorical perspective encourages us to focus not on the internal structure of objects but on the morphisms between them—the structure-preserving relationships. This shift in emphasis proves crucial: in deep learning, what matters is not the particular representation of a vector space but how information flows through transformations.[10]

2.2 Functors: Structure-Preserving Maps

Just as morphisms are structure-preserving maps between objects within a category, *functors* are structure-preserving maps between categories themselves. They form the backbone of categorical thinking, enabling us to translate problems from one domain to another whilst preserving essential relationships.

[10]This philosophy echoes the "structure, not stuff" mantra articulated by John Baez and others in the applied category theory community.

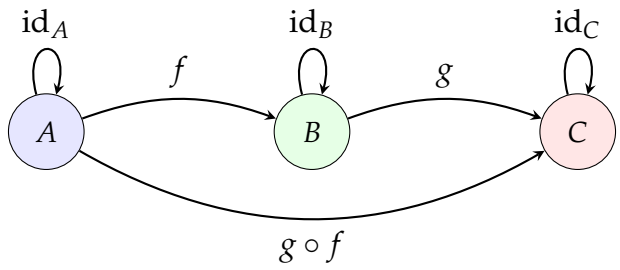


Figure 3: Basic categorical structure showing objects, morphisms, composition ($g \circ f$), and identity morphisms. Composition is read right-to-left: first apply f, then g.

Definition 2.2 (Functor)**.** A *functor* $F : \mathcal{C} \to \mathcal{D}$ between categories consists of:

- An object mapping: for each $A \in \mathrm{ob}(\mathcal{C})$, an object $F(A) \in \mathrm{ob}(\mathcal{D})$;
- A morphism mapping: for each $f : A \to B$ in $\mathcal{C}$, a morphism $F(f) : F(A) \to F(B)$ in $\mathcal{D}$;

satisfying:

1. **Preservation of identities**: $F(\mathrm{id}_A) = \mathrm{id}_{F(A)}$ for all objects A;
2. **Preservation of composition**: $F(g \circ f) = F(g) \circ F(f)$ for all composable morphisms.

Functors appear throughout deep learning in various guises. Consider the forgetful functor $U : \mathbf{Vect}_{\mathbb{R}} \to \mathbf{Set}$ that sends each vector space to its underlying set and each linear map to its underlying function—this "forgets" the linear structure. Conversely, the free functor $F : \mathbf{Set} \to \mathbf{Vect}_{\mathbb{R}}$ sends each set S to the vector space with basis S.[11]

Example 2.2 (The Neural Network Functor)**.** Consider a neural network layer as a parameterised function $f_\theta : \mathbb{R}^n \to \mathbb{R}^m$. Fixing the architecture whilst varying parameters defines a functor from a "parameter space category" to the category of Euclidean spaces and smooth maps. This perspective, developed by Cruttwell et al. (2022), forms the basis of the Para construction discussed in Section 3.

2.3 Natural Transformations: Morphisms Between Functors

If functors are structure-preserving maps between categories, *natural transformations* are structure-preserving maps between functors. They capture the notion of a systematic, coherent way of transforming one construction into another.

[11] The pair (F, U) forms an *adjunction*, a fundamental categorical relationship we discuss in Section 2.5.

$$\begin{tikzcd} A \arrow[r, "f"] \arrow[d, "F"'] & B \arrow[d, "F"] \\ F(A) \arrow[r, "F(f)"'] & F(B) \end{tikzcd}$$

Figure 4: A functor F maps objects and morphisms from one category to another, preserving the compositional structure. The diagram commutes: following either path from A to $F(B)$ yields the same result.

Definition 2.3 (Natural Transformation)**.** Given functors $F, G : \mathcal{C} \to \mathcal{D}$, a *natural transformation* $\alpha : F \Rightarrow G$ consists of:

- For each object $A \in \mathcal{C}$, a morphism $\alpha_A : F(A) \to G(A)$ in $\mathcal{D}$, called the *component* of α at A;

such that for every morphism $f : A \to B$ in $\mathcal{C}$, the following diagram commutes:

$$G(f) \circ \alpha_A = \alpha_B \circ F(f) \tag{1}$$

The naturality condition (1) is powerful: it ensures that the transformation α is compatible with the structure of the categories involved. In machine learning terms, naturality often corresponds to *equivariance*—the property that a transformation commutes with certain operations.[12]

$$\begin{tikzcd} F(A) \arrow[r, "\alpha_A"] \arrow[d, "F(f)"'] & G(A) \arrow[d, "G(f)"] \\ F(B) \arrow[r, "\alpha_B"'] & G(B) \end{tikzcd}$$

Figure 5: The naturality square for a natural transformation $\alpha : F \Rightarrow G$. Commutativity means that transforming then mapping equals mapping then transforming.

2.4 Monoidal Categories and String Diagrams

Deep learning architectures exhibit a distinctive kind of composition: not merely sequential (layer after layer) but also parallel (multiple pathways processing information simultaneously). *Monoidal categories* capture this dual composition structure.[13]

[12]For instance, a pooling operation that is natural with respect to permutations is precisely a permutation-equivariant pooling.

[13]Monoidal categories were introduced by Mac Lane (1963) and have since become fundamental to categorical quantum mechanics, linguistics, and now machine learning.

Definition 2.4 (Monoidal Category)**.** A *monoidal category* $(\mathcal{C}, \otimes, I)$ consists of:

- A category $\mathcal{C}$;
- A bifunctor $\otimes : \mathcal{C} \times \mathcal{C} \to \mathcal{C}$ called the *tensor product*;
- A *unit object* $I \in \mathcal{C}$;
- Natural isomorphisms for associativity and unit laws, satisfying coherence conditions.

The tensor product $\otimes$ captures parallel composition: if $f : A \to B$ and $g : C \to D$, then $f \otimes g : A \otimes C \to B \otimes D$ represents applying f and g in parallel. Combined with sequential composition $\circ$, this yields a rich algebraic structure perfectly suited to describing neural network architectures.

String diagrams provide a powerful graphical calculus for monoidal categories.[14] In this notation, objects are represented by wires and morphisms by boxes. Sequential composition corresponds to vertical stacking (connecting outputs to inputs), whilst tensor products correspond to horizontal juxtaposition.

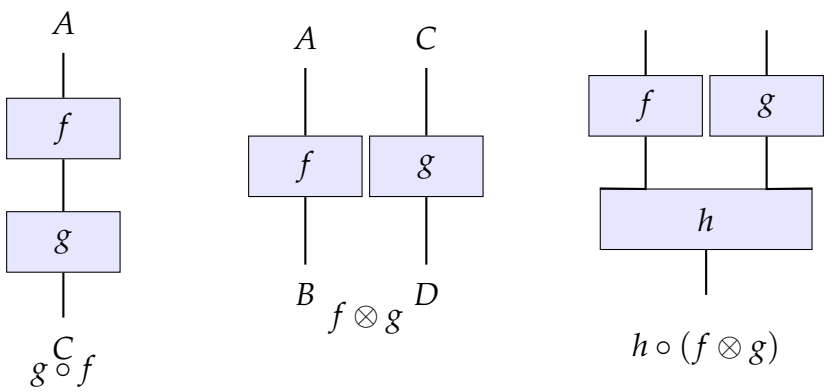


Figure 6: String diagrams for monoidal categories. Left: sequential composition (vertical). Centre: parallel composition via tensor product (horizontal). Right: combining both forms of composition, as commonly occurs in neural network architectures.

String diagrams offer more than notational convenience. The *coherence theorem* for monoidal categories guarantees that any two ways of parsing a string diagram that look topologically equivalent are indeed equal.[15] This graphical reasoning capability makes string diagrams invaluable for verifying architectural equivalences and deriving optimisations.

[14]String diagrams were developed by Joyal and Street (1991) and popularised in physics and computer science contexts by Selinger (2010).

[15]This was proved by Mac Lane (1963) for monoidal categories and extended to various enriched settings by subsequent authors.

2.5 Adjunctions and Universal Properties

Adjunctions represent one of the most important concepts in category theory, capturing optimal relationships between categories. An adjunction consists of two functors going in opposite directions, related by a natural correspondence.

Definition 2.5 (Adjunction)**.** An *adjunction* between categories $\mathcal{C}$ and $\mathcal{D}$ consists of:

- Functors $F : \mathcal{C} \to \mathcal{D}$ (the *left adjoint*) and $G : \mathcal{D} \to \mathcal{C}$ (the *right adjoint*);
- A natural isomorphism $\mathcal{D}(F(A), B) \cong \mathcal{C}(A, G(B))$ for all objects $A \in \mathcal{C}$ and $B \in \mathcal{D}$.

We write $F \dashv G$ to denote that F is left adjoint to G.

Adjunctions appear throughout mathematics and computer science, capturing notions of "free constructions," "optimal approximations," and "Galois connections."[16] In deep learning contexts, adjunctions relate to optimisation (finding best approximations), representation learning (constructing universal representations), and the relationship between forward and backward passes.[17]

$$\begin{tikzcd} \mathcal{C} \arrow[r, bend left, "F"] \arrow[r, phantom, "\perp"] & \mathcal{D} \arrow[l, bend left, "G"] \end{tikzcd}$$

Figure 7: An adjunction between categories $\mathcal{C}$ and $\mathcal{D}$, with F the left adjoint and G the right adjoint.

2.6 The Yoneda Lemma and Representability

The Yoneda lemma, often described as the most important result in category theory, states that an object is completely determined by its relationships to all other objects.[18] This principle has profound implications for representation learning.

Theorem 2.1 (Yoneda Lemma)**.** *For any functor $F : \mathcal{C}^{\mathrm{op}} \to \mathbf{Set}$ and object $A \in \mathcal{C}$:*

$$Nat(\mathcal{C}(-, A), F) \cong F(A)$$

where $\mathcal{C}(-, A)$ denotes the representable functor and Nat denotes natural transformations.

[16] The free-forgetful adjunction between groups and sets, the product-exponential adjunction in cartesian closed categories, and the tensor-hom adjunction in linear algebra are all instances.

[17] Gavranović (2024) argues that many deep learning constructions arise from categorical universal properties, which are in turn captured by adjunctions.

[18] The Yoneda lemma was proved by Nobuo Yoneda during a conversation with Saunders Mac Lane at a Paris café in 1954. See Mac Lane (2013) for historical notes.

The Yoneda lemma implies that studying an object through its morphisms (the "relational" view) is equivalent to studying the object directly. For representation learning, this suggests that learned representations should capture how objects relate to each other, not merely intrinsic properties.[19]

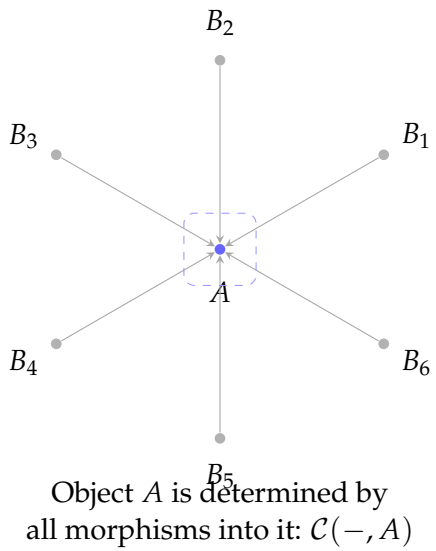


Figure 8: Visualisation of the Yoneda perspective: an object A is fully characterised by the collection of all morphisms into it from other objects.

3 Compositional Foundations of Deep Learning

Neural networks are, at their core, compositions of simpler functions arranged in intricate architectures. This section examines categorical frameworks for understanding this compositional structure, focusing on how category theory provides both a rigorous semantics and new architectural insights.[20]

The principle of compositionality—that the meaning of a complex expression is determined by the meanings of its parts and the rules for combining them—has ancient roots in logic and linguistics. In the context of neural networks, compositionality manifests in multiple ways: layers compose to form deep networks, modules compose to form complex architectures, and learned representations compose to capture hierarchical structure in data. Category theory, as the mathematics of composition, provides the ideal framework for understanding and exploiting these compositional structures.

[19]This connection has been explored by Bradley (2020) and Shiebler (2021), who relate the Yoneda perspective to embedding methods and attention mechanisms.

[20]The compositional view has been championed by the Topos Institute and collaborators; see Fong and Spivak (2019) for an accessible treatment.

3.1 Neural Networks as Morphisms

The simplest categorical view of neural networks identifies them with morphisms in an appropriate category. A feedforward neural network $f : \mathbb{R}^n \to \mathbb{R}^m$ is a morphism in the category **Euc** of Euclidean spaces and smooth functions. This immediately grants access to categorical tools: composition corresponds to stacking layers, and identities correspond to pass-through connections.

However, this naive view misses crucial structure. Neural networks are not arbitrary smooth functions—they are *parameterised* families of functions, with training selecting parameters to optimise performance. Capturing this parameterisation requires more sophisticated categorical machinery.

Definition 3.1 (Parameterised Morphism)**.** A *parameterised morphism* from A to B with parameter space P is a morphism $f : P \times A \to B$ in some category $\mathcal{C}$. We write $f_p : A \to B$ for the morphism obtained by fixing $p \in P$.

3.2 The Para Construction

The *Para construction*, developed by Cruttwell et al. (2022) building on earlier work, provides an elegant categorical framework for parameterised morphisms.[21] It constructs a new category $\mathbf{Para}(\mathcal{C})$ from a base category $\mathcal{C}$ equipped with suitable structure.

Definition 3.2 (Para Construction)**.** Given a monoidal category $(\mathcal{C}, \otimes, I)$, the category $\mathbf{Para}(\mathcal{C})$ has:

- **Objects**: Same as $\mathcal{C}$;
- **Morphisms**: A morphism $A \to B$ in $\mathbf{Para}(\mathcal{C})$ is a pair (P, f) where $P \in \mathcal{C}$ and $f : P \otimes A \to B$ is a morphism in $\mathcal{C}$;
- **Composition**: Given $(P, f) : A \to B$ and $(Q, g) : B \to C$, their composition is $(P \otimes Q, g \circ (\mathrm{id}_Q \otimes f))$;
- **Identity**: $\mathrm{id}_A = (I, \lambda_A)$ where $\lambda_A : I \otimes A \to A$ is the left unitor.

The Para construction captures how neural network composition works in practice: when composing two parameterised layers, the parameters accumulate via the tensor product.[22]

[21] The name "Para" derives from "parameterised." Related constructions appear in the work of Hermida and Jacobs (2000) on fibrations and Capucci and Gavranović (2022) on actegories.

[22] In concrete terms, if layer 1 has weight matrix W_1 and layer 2 has W_2, the composed network has parameters (W_1, W_2), which is precisely the tensor product of parameter spaces.

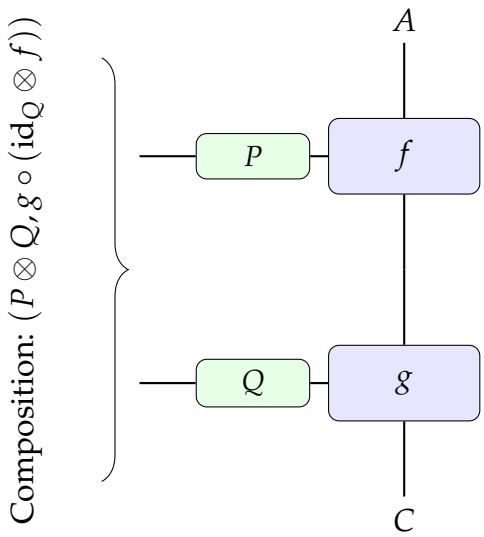


Figure 9: The Para construction: composing parameterised morphisms. Parameters P and Q combine via tensor product, whilst the underlying functions compose sequentially.

The Para construction has proven remarkably useful for understanding deep learning architectures. Gavranović (2024) demonstrates that many standard architectural patterns—skip connections, attention mechanisms, normalisation layers—have elegant descriptions in this framework.

3.3 String Diagrams for Neural Architectures

The monoidal structure underlying neural networks makes them amenable to string diagram representation. This graphical language has been extensively developed for neural networks by Fong et al. (2019) and subsequent authors.[23]

A feedforward network with multiple layers can be drawn as a string diagram, with parallel pathways represented by horizontal juxtaposition and sequential processing by vertical stacking. This representation immediately reveals architectural properties: parallel paths, shared parameters, residual connections, and attention patterns all have distinctive graphical signatures.

[23]The use of string diagrams for neural networks was anticipated by Penrose (1971) in the context of tensor networks, which have deep connections to neural architectures.

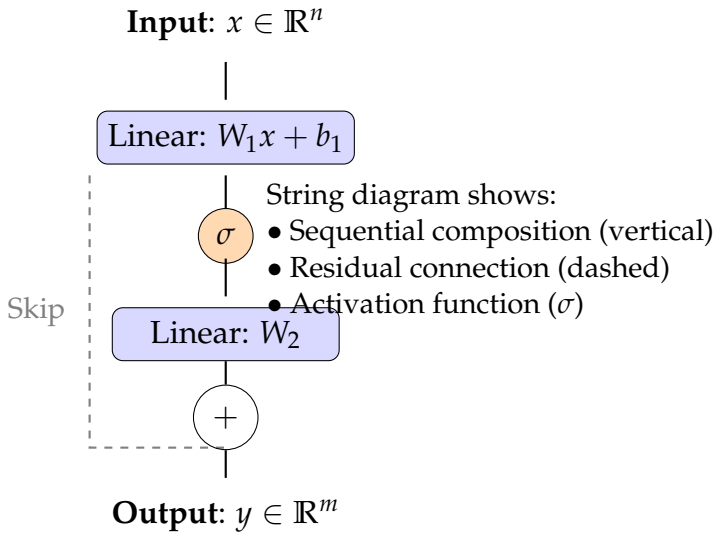


Figure 10: A simple residual network block as a string diagram. The dashed line represents the skip connection—a distinctive feature of ResNets (He et al., 2016)—naturally depicted in the string diagram calculus.

3.4 Polynomial Functors for Architecture Specification

Polynomial functors provide a powerful framework for specifying and reasoning about neural network architectures.[24] A polynomial functor encodes the "shape" of an architecture, specifying input/output types and how components connect.

Definition 3.3 (Polynomial Functor)**.** A *polynomial functor* $p : \mathbf{Set} \to \mathbf{Set}$ is one of the form:

$$p(Y) = \sum_{i \in I} Y^{A_i}$$

where I is a set of "positions" and each A_i is a set of "directions" from position i.

Polynomial functors can encode complex architectural patterns. For instance, a transformer attention head can be specified as a polynomial functor mapping query, key, and value spaces to output representations.[25]

Example 3.1 (Attention as Polynomial Functor)**.** An attention mechanism with n input positions and embedding dimension d corresponds to a polynomial functor of the form:

$$\mathrm{Attn}(Y) = \prod_{i=1}^{n} Y^{\mathbb{R}^d \times \mathbb{R}^d \times \mathbb{R}^d}$$

where the three copies of $\mathbb{R}^d$ in the exponent represent query, key, and value vectors.

[24] Polynomial functors in this sense were developed systematically by Kock (2012) and applied to dynamical systems and machine learning by Spivak (2020).

[25] This perspective is developed by Spivak (2021), who show how polynomial functors capture the compositional structure of attention mechanisms.

3.5 Operads and Wiring Diagrams

For more complex compositional patterns, *operads* provide the appropriate categorical structure.[26] An operad encodes operations with multiple inputs and one output, along with rules for how these operations compose.

Definition 3.4 (Operad)**.** An *operad* $\mathcal{O}$ consists of:

- For each $n \geq 0$, a set $\mathcal{O}(n)$ of n-ary operations;
- Composition maps $\gamma : \mathcal{O}(n) \times \mathcal{O}(k_1) \times \cdots \times \mathcal{O}(k_n) \to \mathcal{O}(k_1 + \cdots + k_n)$;
- An identity element $\mathrm{id} \in \mathcal{O}(1)$;

satisfying associativity, unit, and equivariance axioms.

Wiring diagrams, developed by Spivak (2013), provide a graphical language for operadic composition. In this framework, neural network architectures are described by how components "wire together," with operadic composition capturing hierarchical assembly.[27]

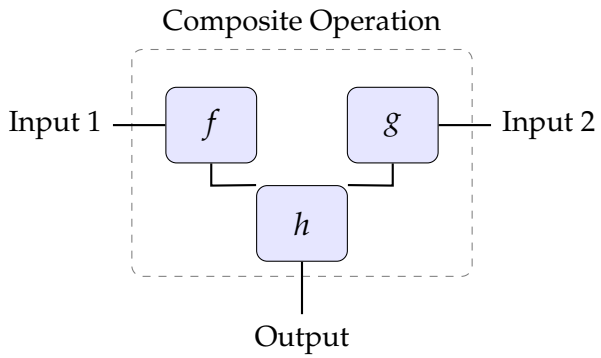


Figure 11: A wiring diagram showing operadic composition. The outer dashed box represents a composite operation built from inner components f, g, and h. External interfaces define how the composite connects to its environment.

The operadic perspective is particularly valuable for neural architecture search and meta-learning, where we wish to reason about spaces of architectures rather than individual networks. Operads provide a principled way to specify what compositions are valid, guiding search towards architecturally coherent designs.

[26] Operads originated in algebraic topology with May (1972) and have since found applications across mathematics, physics, and computer science. For an accessible introduction, see Leinster (2004).

[27] The wiring diagram approach has been applied to neural architecture search by Niu et al. (2019), who use operadic structure to guide the search over architectural spaces.

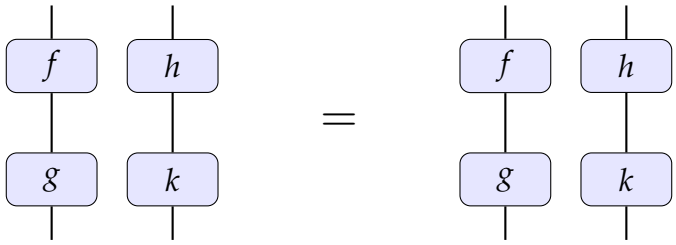
3.6 Symmetric Monoidal Categories and the Interchange Law

A crucial property of neural network composition is the *interchange law*, which states that the order of sequential and parallel composition does not matter when both are applicable. This property is captured by *symmetric monoidal categories*.[28]

Proposition 3.1 (Interchange Law)**.** *In a strict monoidal category, for morphisms* $f : A \to B$, $g : B \to C$, $h : A' \to B'$, $k : B' \to C'$*:*

$$(g \circ f) \otimes (k \circ h) = (g \otimes k) \circ (f \otimes h)$$

This law ensures that string diagrams are well-defined: we can slide boxes past each other horizontally without changing the computed result, provided there are no wire crossings. For neural networks, this means that the order in which we conceptualise parallel computations is immaterial to the final output—a fundamental property underlying parallel execution on GPUs.[29]



Interchange law: order of sequential/parallel composition is immaterial

Figure 12: The interchange law in string diagrams. The two sides represent the same computation, illustrating that parallel and sequential composition commute when applicable.

3.7 Compact Closed Categories and Duality

Some neural network architectures exhibit *duality* structure—the ability to "turn inputs into outputs" and vice versa. This is captured by *compact closed categories*, which extend monoidal categories with dual objects.[30]

Definition 3.5 (Compact Closed Category)**.** A *compact closed category* is a symmetric monoidal category where every object A has a *dual* A^* equipped with:

[28]Symmetric monoidal categories add a symmetry natural isomorphism $\sigma_{A,B} : A \otimes B \to B \otimes A$ satisfying coherence conditions.

[29]Modern deep learning frameworks exploit this parallelism automatically, but the categorical perspective makes explicit the algebraic conditions under which such parallelisation is valid.

[30]Compact closed categories are central to categorical quantum mechanics (Coecke and Kissinger, 2017) and arise naturally in linear algebra, where vector spaces have duals.

- An *evaluation* morphism $\epsilon_A : A^* \otimes A \to I$;
- A *coevaluation* morphism $\eta_A : I \to A \otimes A^*$;

satisfying the "snake equations" (yanking equations).

In the string diagram calculus, dual objects allow wires to bend backwards, representing the flow of information in the "opposite direction." For neural networks, this duality underlies the relationship between the forward pass (computing outputs from inputs) and the backward pass (propagating gradients from outputs to inputs).[31]

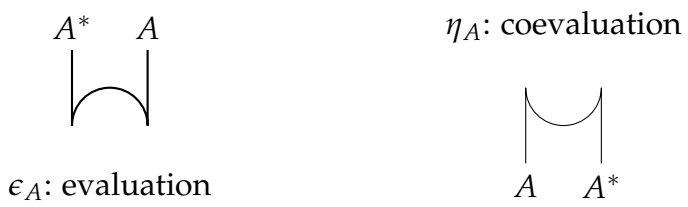


Figure 13: The cup (evaluation) and cap (coevaluation) morphisms in a compact closed category. These allow wires to "bend," creating duality between objects and their duals.

3.8 Traced Monoidal Categories and Feedback

Neural networks with feedback connections—such as recurrent networks—require categorical structures that can represent loops. *Traced monoidal categories*, introduced by Joyal and Street (1996), provide exactly this capability.[32]

Definition 3.6 (Trace)**.** A *trace* on a symmetric monoidal category $\mathcal{C}$ is a family of operations

$$\mathrm{Tr}^U_{A,B} : \mathcal{C}(A \otimes U, B \otimes U) \to \mathcal{C}(A, B)$$

satisfying naturality, superposing, yanking, and other coherence axioms.

For recurrent neural networks, the trace captures how hidden state circulates through time steps. The traced monoidal category framework ensures that such feedback structures compose correctly and provides algebraic tools for reasoning about their behaviour.[33]

[31] This connection is made precise in the lens formalism of Section 4.3, where forward and backward passes form dual aspects of a single structure.

[32] A trace operation allows a portion of a morphism's output to be fed back as input, creating a loop in the string diagram.

[33] The connection between traces and fixpoint operators provides a categorical perspective on the equilibrium states of recurrent networks.

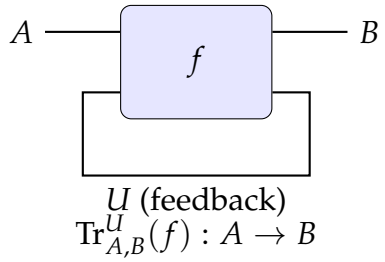


Figure 14: A traced morphism: the output on wire U is fed back as input, creating a loop. This structure models feedback connections in recurrent neural networks.

4 Categorical Perspectives on Differentiation and Learning

Gradient-based optimisation lies at the heart of deep learning. This section examines how category theory illuminates the structure of differentiation and learning, providing both theoretical insights and practical algorithms.[34]

4.1 Cartesian Differential Categories

Cartesian differential categories, introduced by Blute et al. (2009), axiomatise the properties of differentiation in a purely categorical setting. They provide the foundation for understanding automatic differentiation and gradient computation.[35]

Definition 4.1 (Cartesian Differential Category)**.** A *Cartesian differential category* is a Cartesian left-additive category $\mathcal{C}$ equipped with a *differential combinator*

$$D : \mathcal{C}(A, B) \to \mathcal{C}(A \times A, B)$$

satisfying seven axioms encoding the properties of differentiation.

The key insight is that differentiation can be understood as a systematic transformation of morphisms: the differential combinator D takes a morphism $f : A \to B$ and produces its "derivative" $D[f] : A \times A \to B$, which is linear in the second argument.[36]

[34]The categorical study of differentiation has a long history; see Blute et al. (2009) for foundational work. Application to deep learning is more recent, with key contributions by Fong et al. (2019) and Cruttwell et al. (2022).

[35]The axioms capture essential properties like the chain rule, product rule, and linearity of the derivative in the second argument.

[36]The two copies of A in the domain represent the point at which the derivative is evaluated and the tangent direction.

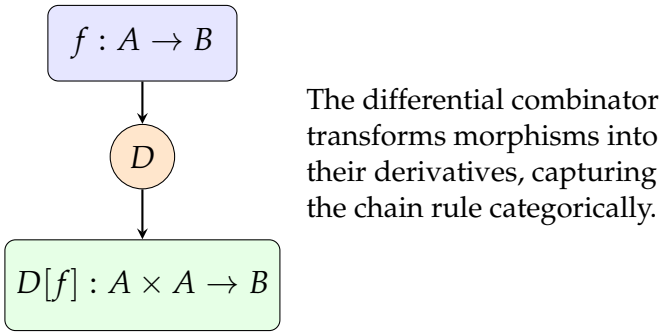


Figure 15: The differential combinator D transforms a morphism f into its derivative $D[f]$. This categorical abstraction unifies various notions of differentiation.

4.2 Reverse Derivative Categories and Backpropagation

Whilst Cartesian differential categories capture forward-mode differentiation, deep learning relies on *reverse-mode* differentiation (backpropagation). Cockett and Cruttwell (2020) introduced *reverse derivative categories* to axiomatise this dual perspective.

Definition 4.2 (Reverse Derivative Category)**.** A *reverse derivative category* is a Cartesian left-additive category $\mathcal{C}$ equipped with a *reverse differential combinator*

$$R : \mathcal{C}(A, B) \to \mathcal{C}(A \times B, A)$$

satisfying axioms dual to those of Cartesian differential categories.

The reverse derivative $R[f] : A \times B \to A$ takes a point $a \in A$ and a "cotangent" $b^* \in B^*$ (represented as an element of B via inner product structure) and produces the gradient $\nabla_a f \cdot b^*$.[37]

Proposition 4.1 (Cockett and Cruttwell, 2020)**.** *In a reverse derivative category with forward derivatives, the chain rule for reverse differentiation holds:*

$$R[g \circ f] = R[f] \circ (\mathrm{id} \times R[g]) \circ (\mathrm{id} \times (f \times \mathrm{id})) \circ \Delta$$

where Δ is the diagonal and operations compose appropriately.

This categorical chain rule precisely captures the structure of backpropagation: to compute the gradient through a composition $g \circ f$, we first propagate gradients through g, then through f, accumulating via the chain rule.

[37] In neural network terminology, b^* is the "upstream gradient" and $\nabla_a f \cdot b^*$ is the gradient backpropagated through f.

4.3 Lenses and Optics for Bidirectional Data Flow

The *lens* formalism, originating in functional programming as a pattern for bidirectional data access (Foster et al., 2007), provides another categorical perspective on backpropagation.[38]

Definition 4.3 (Lens)**.** A *lens* from A to B in a Cartesian category consists of:

- A *get* function: $\text{get} : A \to B$
- A *put* function: $\text{put} : A \times B \to A$

subject to coherence laws ensuring get and put interact appropriately.

For neural networks, the "get" function is the forward pass, computing the output from the input. The "put" function represents the backward pass, updating the input (or parameters) given the output gradient. The lens laws ensure that forward and backward passes are consistent.[39]

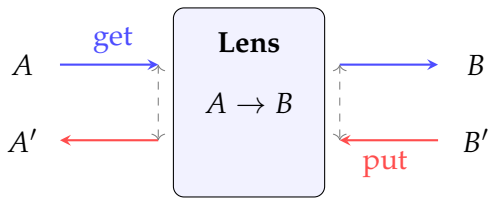


Figure 16: A lens capturing bidirectional data flow. The "get" direction (blue, top) corresponds to the forward pass; the "put" direction (red, bottom) corresponds to the backward pass. Primes indicate updated/gradient values.

Theorem 4.1 (Fong et al., 2019)**.** *The category* **Learn** *of learners—parameterised functions equipped with forward passes, backward passes, and update rules—is equivalent to a category of lenses in a suitable monoidal category.*

This theorem establishes that the compositional structure of deep learning (stacking layers, training end-to-end) is captured precisely by lens composition. It provides a foundation for understanding why gradient-based learning "works" compositionally and suggests avenues for generalisation.[40]

[38]The connection between lenses and backpropagation was made explicit by Fong et al. (2019), who showed that neural networks with their backward passes form a category of lenses.

[39]The lens framework generalises to *optics*, which handle a broader class of bidirectional transformations; see Riley (2018) for the categorical treatment.

[40]Extensions include *dependent lenses* for architectures with dynamic shapes and *polynomial lenses* for richer feedback structures.

4.4 Gradient Descent as a Functor

Viewing gradient descent through a categorical lens reveals it as a *functor* from a category of optimisation problems to a category of dynamical systems.[41]

Example 4.1 (Gradient Descent Functor)**.** Consider a category **Opt** whose objects are tuples (P, L) of a parameter space P and a loss function $L : P \to \mathbb{R}$, and whose morphisms are structure-preserving maps between optimisation problems. Gradient descent defines a functor

$$\mathrm{GD} : \mathbf{Opt} \to \mathbf{Dyn}$$

to a category of dynamical systems, sending (P, L) to the dynamical system $\dot{p} = -\nabla L(p)$.

This functorial view clarifies the relationship between the structure of an optimisation problem and the dynamics of its solution. Functoriality implies that composing optimisation problems (e.g., multi-task learning) corresponds to composing their gradient flows in a predictable way.

4.5 The Seven Axioms of Cartesian Differential Categories

The formal axiomatisation of differentiation in Cartesian differential categories comprises seven axioms that capture the essential properties of the derivative operation. Understanding these axioms provides insight into what makes differentiation compositional and why it interacts well with categorical structure.[42]

Let $D[f]$ denote the derivative of $f : A \to B$, viewed as a morphism $D[f] : A \times A \to B$ where the first argument is the point and the second is the tangent direction.

1. **CD-1 (Constant Rule)**: $D[c \circ \pi_1] = 0$ for any constant c—the derivative of a constant is zero.

2. **CD-2 (Linearity)**: $D[f]$ is linear (additive and homogeneous) in its second argument—derivatives are linear maps.

3. **CD-3 (Identity)**: $D[\mathrm{id}_A] = \pi_2$—the derivative of the identity is projection onto the tangent direction.

4. **CD-4 (Pairing)**: $D[\langle f, g\rangle] = \langle D[f], D[g]\rangle$—the derivative of a pair is the pair of derivatives.

[41] This perspective is developed by Cruttwell et al. (2022) and connects to broader themes in categorical dynamics; see Spivak (2019) for related ideas.

[42] These axioms were carefully chosen by Blute et al. (2009) to be both sufficient for deriving standard calculus results and applicable to non-classical settings like synthetic differential geometry.

5. **CD-5 (Projection)**: $D[\pi_i] = \pi_i \circ \pi_2$—the derivative of a projection is projection on the tangent component.

6. **CD-6 (Chain Rule)**: $D[g \circ f] = D[g] \circ \langle f \circ \pi_1, D[f] \rangle$—composition differentiates via the chain rule.

7. **CD-7 (Symmetry)**: $D[D[f]] \circ c = D[D[f]]$ where c swaps middle components—second derivatives are symmetric.

The chain rule (CD-6) is perhaps the most crucial for deep learning: it asserts that the derivative of a composition can be computed by composing local derivatives. This is precisely the mathematical foundation of backpropagation.[43]

$$\begin{tikzcd} A \times A \arrow[r, "f \times D[f]"] \arrow[d, "D[g \circ f]"'] & B \times B \arrow[d, "D[g]"] \\ C & C \end{tikzcd}$$

Figure 17: The chain rule as a commutative diagram: $D[g \circ f]$ factors through $D[g]$ and $D[f]$, making backpropagation a consequence of categorical structure.

4.6 Automatic Differentiation Modes

Automatic differentiation (AD) can be performed in two fundamental modes: *forward mode* and *reverse mode*. The categorical perspective reveals these as dual constructions.[44]

Forward Mode AD implements the differential combinator D directly, propagating tangent vectors alongside function values. Given $f : \mathbb{R}^n \to \mathbb{R}^m$ and a tangent direction $v \in \mathbb{R}^n$, forward mode computes the directional derivative $D[f](x, v) = \nabla f(x) \cdot v$ in time $O(\text{cost of } f)$. Computing the full Jacobian requires n passes.[45]

Reverse Mode AD implements the reverse derivative combinator R, propagating cotangent vectors backward through the computation. Given $f : \mathbb{R}^n \to \mathbb{R}^m$ and a cotangent $w \in \mathbb{R}^m$, reverse mode computes $R[f](x, w) = w^T \cdot \nabla f(x)$ in time $O(\text{cost of } f)$. Computing the full Jacobian requires m passes.[46]

[43] In neural network terms, CD-6 states that the gradient through a composition $g \circ f$ is computed by applying f in the forward direction, computing the local derivative of g, and multiplying by the local derivative of f.

[44] Forward mode computes directional derivatives; reverse mode computes gradients. Their computational complexity differs based on the relative dimensions of input and output spaces.

[45] Forward mode is efficient when $n \ll m$—few inputs and many outputs—which is rare in deep

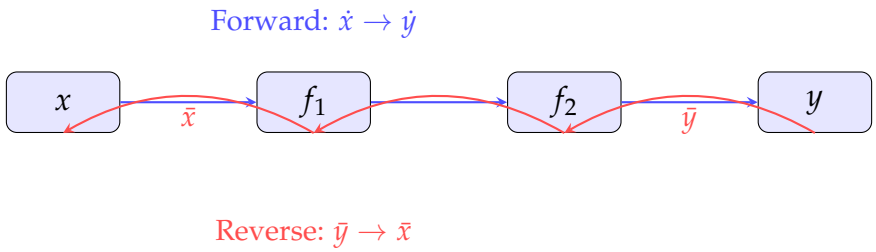


Figure 18: Forward mode AD propagates tangent vectors ($\dot{x}$) forward through the computation graph; reverse mode propagates cotangent vectors ($\bar{y}$) backward. Reverse mode is the categorical dual.

The categorical duality between forward and reverse mode AD corresponds to the duality between tangent and cotangent bundles in differential geometry. Reverse derivative categories (Cockett and Cruttwell, 2020) axiomatise the reverse mode directly, providing foundations for understanding and implementing backpropagation.[47]

4.7 Loss Landscapes and Categorical Topology

The geometry of loss landscapes—the surfaces defined by loss functions over parameter spaces—profoundly influences training dynamics. Categorical perspectives, particularly through *sheaf theory*, offer new tools for understanding these landscapes.[48]

A sheaf assigns data (e.g., local loss values, gradients, or curvature information) to open sets of parameter space in a way that respects how the sets overlap. This provides a framework for studying how local properties of the loss landscape combine to determine global training behaviour.

5 Geometry, Symmetry, and Equivariant Neural Networks

Modern deep learning increasingly recognises the importance of incorporating geometric and symmetry structure into neural architectures. This section examines categorical perspectives on *geometric deep learning*—the programme of designing neural networks that respect the symmetries of their input domains.[49]

learning.

[46]Reverse mode is efficient when $m \ll n$—many inputs and few outputs. Since neural networks typically have scalar loss functions ($m = 1$), reverse mode dominates deep learning.

[47]The `ad` library in Haskell implements both modes using a uniform categorical interface based on these principles.

[48]Sheaf-theoretic approaches to data analysis were pioneered by Robinson (2014); applications to neural network loss landscapes are more speculative but promising.

[49]The geometric deep learning framework was systematised by Bronstein et al. (2021), who unified diverse architectures (CNNs, GNNs, Transformers) through the lens of symmetry and equivariance.

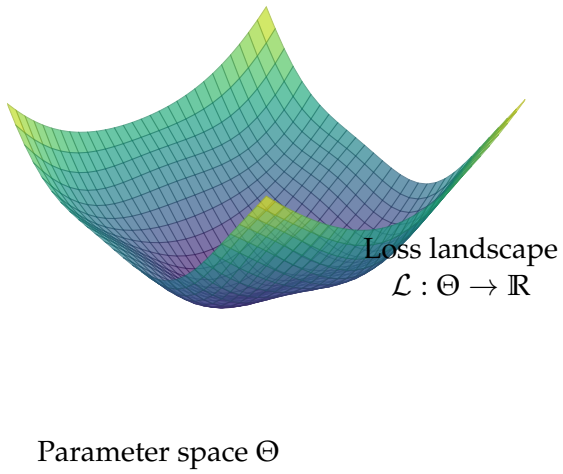


Figure 19: A loss landscape visualisation. Categorical and sheaf-theoretic tools can analyse how local curvature and gradient information across the parameter space determines global optimisation dynamics.

5.1 Groups, Representations, and Functor Categories

The foundation of geometric deep learning is *group equivariance*: designing networks whose outputs transform predictably when inputs are transformed by group actions. Category theory provides the natural language for this structure through *functor categories*.[50]

A group G can be viewed as a category with a single object and morphisms corresponding to group elements. A *G-representation* on a vector space V is then a functor $\rho : G \to \mathbf{Vect}$.[51]

Definition 5.1 (Equivariant Map)**.** Given G-representations $\rho : G \to \mathbf{Vect}$ on V and $\sigma : G \to \mathbf{Vect}$ on W, an *equivariant map* (or *intertwiner*) is a linear map $f : V \to W$ such that for all $g \in G$:

$$f \circ \rho(g) = \sigma(g) \circ f$$

This is precisely a natural transformation between the functors ρ and σ.

This categorical perspective immediately reveals that the space of equivariant neural network layers is characterised by *natural transformations* between representation functors. Finding equivariant architectures thus reduces to computing spaces of intertwiners—a well-studied problem in representation theory.[52]

[50]The connection between equivariance and functors is classical in mathematics; its application to neural networks is developed by Cohen and Welling (2016) and subsequent authors.

[51]More precisely, it is a functor to the category of finite-dimensional vector spaces and linear maps.

[52]For finite groups, Schur's lemma and character theory provide computational tools. For Lie groups, the situation is richer and connects to differential geometry.

$$\begin{tikzcd} V \arrow[r, "f"] \arrow[d, "\rho(g)"'] & W \arrow[d, "\sigma(g)"] \\ V \arrow[r, "f"'] & W \end{tikzcd}$$

Figure 20: An equivariant map f intertwines group actions ρ and σ: the diagram commutes for all group elements g.

5.2 Convolutional Networks as Functorial Constructions

Convolutional neural networks (CNNs), the workhorses of image processing, exhibit a specific equivariance structure that admits elegant categorical description.[53]

Proposition 5.1. *A standard convolutional layer $f : L^2(\mathbb{R}^2) \to L^2(\mathbb{R}^2)$ with kernel k satisfies translation equivariance:*

$$f(T_a \cdot x) = T_a \cdot f(x)$$

for all translations T_a, $a \in \mathbb{R}^2$. This is naturality with respect to the translation group action.

More sophisticated group equivariant networks—for rotations, reflections, scale transformations, and beyond—can be systematically constructed using the categorical machinery of induced representations and Mackey's theory.[54]

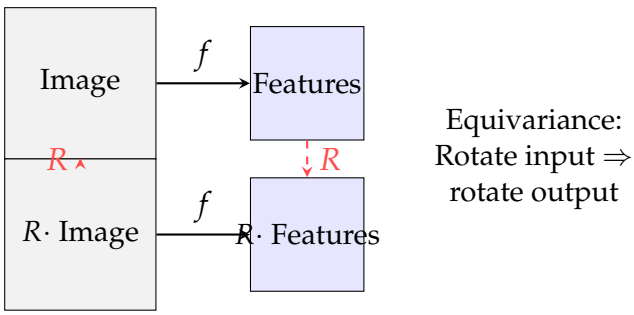


Figure 21: Equivariance in a convolutional network: rotating the input image results in correspondingly rotated feature maps. This commutativity is the hallmark of equivariant architectures.

[53]CNNs are equivariant to translations; this was extended to rotations and other groups by Cohen and Welling (2016) in their seminal work on group equivariant CNNs.

[54]The steerable CNN framework of Cohen et al. (2016) uses this approach to construct rotation-equivariant networks.

5.3 Graph Neural Networks and Sheaf Theory

Graph neural networks (GNNs), which process data defined on graphs, exhibit a richer symmetry structure: permutation equivariance with respect to node reorderings.[55] Categorical perspectives, particularly through *sheaf theory*, provide deep insights into GNN expressivity and design.[56]

Definition 5.2 (Cellular Sheaf)**.** A *cellular sheaf* $\mathcal{F}$ on a graph $G = (V, E)$ assigns:

- To each node $v \in V$, a vector space $\mathcal{F}(v)$ (the *stalk* at v);
- To each edge $e = (u, v) \in E$, linear *restriction maps* $\rho_{v \leftarrow e} : \mathcal{F}(e) \to \mathcal{F}(v)$.

In this framework, a GNN layer corresponds to a morphism of sheaves, propagating information along edges according to the restriction maps. The message-passing paradigm of GNNs is precisely the categorical notion of computing *sheaf cohomology*.[57]

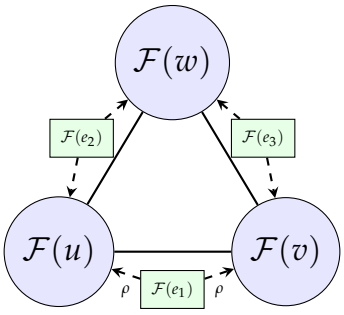


Figure 22: A cellular sheaf on a graph: each node and edge carries a vector space (stalk), with restriction maps specifying how edge data projects to incident nodes. GNN message passing computes with this structure.

5.4 The Yoneda Perspective on Representations

The Yoneda lemma (Section 2.6) offers a profound perspective on representation learning: an object is fully characterised by its relationships to all other objects.[58] This suggests that optimal representations should encode relational information rather than intrinsic properties.

[55] GNNs were introduced by Scarselli et al. (2008) and popularised through architectures like Graph Convolutional Networks (Kipf and Welling, 2017) and GraphSAGE (Hamilton et al., 2017).

[56] The sheaf-theoretic view of GNNs was developed by Hansen and Ghrist (2020) and Bodnar et al. (2022), who use sheaf cohomology to characterise GNN limitations.

[57] This reveals deep connections between GNN expressivity and topological properties of the underlying graph, as explored by Bodnar et al. (2022).

[58] This "relational" view of representation has been connected to embedding methods and kernel methods by Shiebler (2021).

Example 5.1 (Yoneda and Word Embeddings)**.** In word embedding methods like Word2Vec (Mikolov et al., 2013), words are represented by their co-occurrence patterns—their "relationships" to other words. This is precisely the Yoneda perspective: representing an object (word) by the functor of morphisms into it (co-occurrences).

More speculatively, the Yoneda lemma suggests a design principle for representation learning: learn representations that capture how entities relate to each other across the dataset, rather than isolated feature vectors.[59]

5.5 Gauge Equivariance and Fibre Bundles

The most sophisticated geometric deep learning architectures incorporate *gauge equivariance*—invariance to local coordinate changes—through the mathematical framework of *fibre bundles*.[60]

A fibre bundle $E \to M$ consists of a total space E, base space M, and projection map, with "fibres" E_x over each point $x \in M$. Neural networks on fibre bundles must be equivariant to transformations within each fibre (gauge transformations).

Definition 5.3 (Gauge Equivariant Layer)**.** A layer $f : \Gamma(E) \to \Gamma(F)$ between spaces of sections of fibre bundles is *gauge equivariant* if for any gauge transformation g:

$$f(g \cdot s) = g \cdot f(s)$$

where the action of g is defined fibre-wise.

This framework unifies diverse architectures for processing data on manifolds, meshes, and point clouds, providing a principled approach to designing networks that respect the geometric structure of their domains.[61]

5.6 Kan Extensions and Universal Constructions in Representation Learning

Kan extensions, described by Mac Lane as the most important concept in category theory, capture the notion of "optimal approximation" in a categorical setting.[62] Their

[59] This connects to contrastive learning methods, which learn representations by distinguishing related from unrelated pairs.

[60] Gauge theory originates in physics, where it underlies the Standard Model of particle physics. Its application to deep learning was pioneered by Cohen et al. (2019).

[61] Implementations include Gauge Equivariant Mesh CNNs (de Haan et al., 2020) and SE(3)-equivariant networks for molecular modelling (Fuchs et al., 2020).

[62] Mac Lane famously wrote that "All Concepts are Kan Extensions," emphasising their fundamental role.

application to representation learning reveals deep connections between categorical universality and the optimisation objectives of neural networks.

Definition 5.4 (Left Kan Extension)**.** Given functors $F : \mathcal{C} \to \mathcal{E}$ and $K : \mathcal{C} \to \mathcal{D}$, the *left Kan extension* of F along K, denoted $\mathrm{Lan}_K F : \mathcal{D} \to \mathcal{E}$, is a functor equipped with a natural transformation $\eta : F \Rightarrow (\mathrm{Lan}_K F) \circ K$ that is universal among such transformations.

$$\begin{tikzcd} \mathcal{C} \arrow[r, "K"] \arrow[d, "F"'] & \mathcal{D} \arrow[dl, dashed, "\mathrm{Lan}_K F"'] \\ \mathcal{E} & \end{tikzcd}$$

Figure 23: The left Kan extension $\mathrm{Lan}_K F$ extends the functor F along K in a universal way—the "best possible" extension given the available structure.

In representation learning, we can interpret this structure as follows: $\mathcal{C}$ represents a domain of "known" objects (training data), $\mathcal{D}$ represents a larger domain including "unknown" objects (test data), and F assigns representations to known objects. The Kan extension $\mathrm{Lan}_K F$ provides the canonical way to extend these representations to new objects.[63]

Example 5.2 (Kernel Methods as Kan Extensions)**.** In kernel methods, the feature map $\phi : X \to H$ embeds data points into a Hilbert space. The kernel function $k(x, y) = \langle \phi(x), \phi(y) \rangle$ can be viewed as a Kan extension: it extends the inner product structure from finite training data to all of input space in a universal way.

5.7 Limits, Colimits, and Architectural Design

Limits and *colimits* are categorical constructions that capture notions of "aggregation" and "distribution." These structures appear throughout neural network architecture design, providing categorical semantics for common patterns.[64]

Definition 5.5 (Limit)**.** Given a diagram $D : \mathcal{J} \to \mathcal{C}$, the *limit* $\lim D$ is an object equipped with projection morphisms to each object in the diagram, universal among such constructions.

In neural network terms:

[63] This perspective is explored by Shiebler (2021), who connects Kan extensions to inductive biases in learning algorithms.

[64] Products and coproducts (sums) are special cases; more general limits and colimits handle complex aggregation patterns.

- **Products** ($A \times B$) correspond to *concatenation*: combining features from parallel pathways;
- **Equalisers** correspond to *residual connections*: ensuring two pathways produce compatible outputs;
- **Pullbacks** correspond to *attention mechanisms*: selecting features based on compatibility with queries;
- **Coproducts** ($A + B$) correspond to *branching*: routing data along different pathways;
- **Coequalisers** correspond to *pooling*: aggregating features from multiple sources.

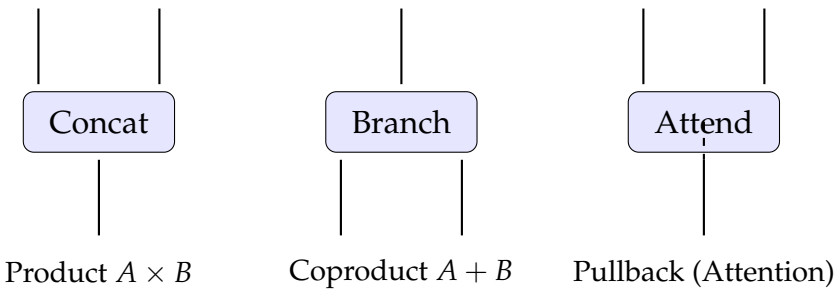


Figure 24: Categorical limits and colimits as architectural primitives: products for concatenation, coproducts for branching, and pullbacks for attention-like selection.

This perspective suggests that architectural innovations can be systematically derived by considering which categorical (co)limits are appropriate for a given task. The universality property of (co)limits ensures that the resulting architectures are, in a precise sense, canonical solutions to the aggregation problems they address.

6 Semantic and Logical Frameworks

Beyond computational structure, category theory offers frameworks for understanding the *meaning* of neural computations. This section examines semantic approaches, including categorical compositional distributional semantics for natural language, type-theoretic foundations, and topos-theoretic perspectives.[65]

[65]The semantic perspective has been championed by the Oxford quantum group and collaborators; see Coecke et al. (2010) for foundational work.

6.1 Categorical Compositional Distributional Semantics

The *DisCoCat* framework (Categorical Compositional Distributional Semantics), developed by Coecke et al. (2010) and Clark et al. (2008), provides a mathematically rigorous approach to compositional semantics for natural language.[66]

Definition 6.1 (DisCoCat Model)**.** A *DisCoCat model* consists of:

- A mapping of grammatical types to vector spaces (objects in **Vect**);
- A mapping of words to vectors or tensors in appropriate spaces;
- Composition via tensor contraction guided by grammatical structure.

The key insight is that grammatical structure, represented via pregroup grammars or similar formalisms, provides the "wiring diagram" for composing word meanings.[67]

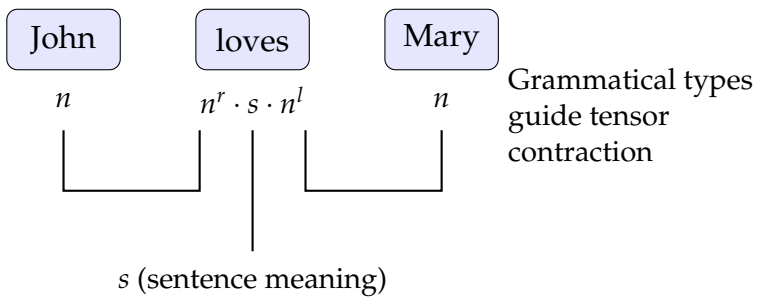


Figure 25: DisCoCat composition for "John loves Mary." The grammatical types (n for noun, s for sentence) determine how word vectors compose via tensor contraction to yield sentence meaning.

DisCoCat has been applied to question answering, sentiment analysis, and other NLP tasks, demonstrating that categorical composition can capture aspects of linguistic meaning.[68] The framework also connects to quantum computing, as the same categorical structures (compact closed categories) underlie quantum protocols.[69]

[66] DisCoCat combines the distributional hypothesis ("you shall know a word by the company it keeps") with categorical composition, yielding a framework where sentence meaning is computed compositionally from word meanings.

[67] Pregroup grammars, introduced by Lambek (2008), are a type-logical formalism for syntax that has a natural categorical interpretation.

[68] See Sadrzadeh et al. (2014) for applications and Kartsaklis et al. (2019) for connections to neural architectures.

[69] This connection has led to "quantum NLP" experiments using quantum computers to implement DisCoCat models (Meichanetzidis et al., 2020).

6.2 Type Systems for Neural Networks

Type systems, foundational to programming language theory, offer another categorical perspective on neural network correctness.[70] A type system for neural networks can ensure that architectures are well-formed and that data flows correctly through layers.

Example 6.1 (Typed Tensor Operations)**.** In a typed framework, a matrix multiplication $W : \mathbb{R}^{m \times n}$ applied to a vector $x : \mathbb{R}^n$ yields a result of type $\mathbb{R}^m$. The type system prevents dimension mismatches at compile time rather than runtime.

More sophisticated type systems can encode equivariance constraints, ensuring that only architectures respecting specified symmetries can be constructed.[71] The categorical semantics of such type systems reveals deep connections between architectural constraints and logical propositions.

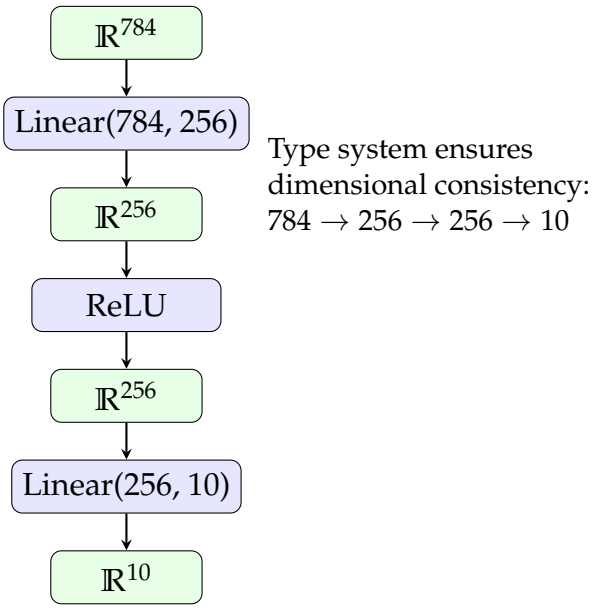


Figure 26: A typed neural network architecture for MNIST digit classification. The type system tracks dimensions through layers, ensuring well-formedness.

6.3 Topos-Theoretic Approaches

Topoi (plural of topos), introduced by Grothendieck (1972) and developed in the work of Lawvere and Rosebrugh (2003), provide extremely general categorical frameworks

[70] The connection between types and categories is the Curry-Howard-Lambek correspondence mentioned in Section 1.

[71] This approach is explored by Shiebler (2020) and implemented in libraries like equinox (Kidger and Garcia, 2021) for JAX.

encompassing both geometry and logic.[72] Their application to neural networks, whilst nascent, offers intriguing possibilities.[73]

Definition 6.2 (Elementary Topos)**.** An *elementary topos* $\mathcal{E}$ is a category with:

- Finite limits (including products and equalisers);
- Exponential objects (function spaces);
- A subobject classifier Ω (generalising the two-element set $\{0, 1\}$).

The subobject classifier Ω in a topos plays the role of "truth values," but these need not be merely true/false—they can have a richer structure. This suggests that topos-theoretic approaches could model uncertainty, probability, or fuzzy reasoning in neural networks.[74]

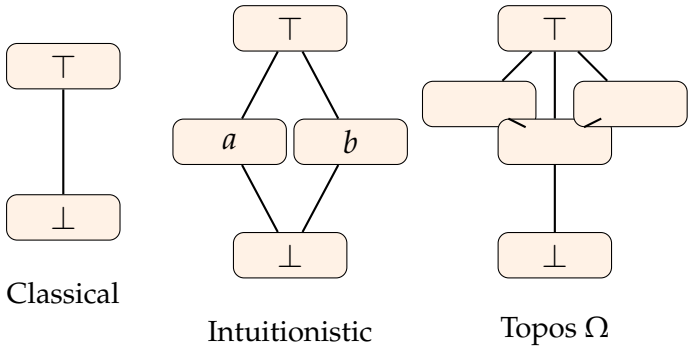


Figure 27: Truth value structures: classical (two values), intuitionistic (Heyting algebra), and general topos (rich lattice structure). Topos-theoretic approaches allow neural networks to operate with generalised logics.

One concrete application area is *contextual semantics*: different contexts (datasets, domains, tasks) might have different "logics" governing valid inferences. Topos theory provides a framework for relating these local logics through *geometric morphisms*—structure-preserving maps between topoi.[75]

6.4 Linear Logic and Resource Sensitivity

Linear logic, introduced by Girard (1987), tracks resource usage—variables must be used exactly once unless explicitly copied or discarded. This connects naturally to

[72] A topos behaves like a "generalised universe of sets" with its own internal logic. Elementary topoi were axiomatised by Lawvere and Rosebrugh (2003).

[73] Early explorations include Shiebler (2021), who discusses topos-theoretic perspectives on learning.

[74] The presheaf topos over a poset, for instance, has a Heyting algebra of truth values corresponding to open sets, enabling reasoning about degrees of truth.

[75] This connects to transfer learning and domain adaptation, where we seek to relate knowledge across different contexts.

neural network computation, where data flows through layers with controlled copying and aggregation.[76]

Definition 6.3 (Key Linear Logic Connectives)**.** Linear logic includes:

- $A \otimes B$: tensor (both A and B available simultaneously);
- $A \multimap B$: linear implication (consume A to produce B);
- $!A$: exponential (unlimited copies of A available).

In a linear type system for neural networks, activations would have linear types (used exactly once in forward propagation), whilst parameters would have exponential types (used throughout the network). This ensures that computational resources—memory for activations, parameters for weight sharing—are tracked correctly.[77]

6.5 Categorical Models of Attention and Transformers

The transformer architecture, with its self-attention mechanism, has become the dominant paradigm for sequence modelling and beyond.[78] Developing categorical models of attention is an active research area, with several promising approaches emerging.

One perspective views attention as a *weighted colimit*: given a set of value vectors, attention computes a weighted combination where the weights depend on query-key compatibility. This can be formalised in categories enriched over probability distributions.[79]

Another approach, developed by Gavranović (2024), embeds transformers within the Para construction by treating attention weights as additional parameters. This perspective clarifies how attention layers compose and how gradients flow through the attention mechanism during training.

6.6 Functorial Semantics for Neural Language Models

Large language models (LLMs) have achieved remarkable capabilities in natural language understanding and generation.[80] Extending DisCoCat-style compositional se-

[76]The connection between linear logic and neural networks is explored by Clift and Murfet (2018), who use linear type systems to reason about autodifferentiation.

[77]This perspective informs the design of efficient automatic differentiation, where tracking linearity enables memory-optimal gradient computation.

[78]Transformers were introduced by Vaswani et al. (2017) and have since achieved state-of-the-art results in NLP, vision, and multimodal learning.

[79]The softmax operation in attention heads produces a probability distribution over positions, suggesting connections to Markov categories (Section 7).

[80]Models like GPT and BERT have transformed NLP; see Brown et al. (2020) for an influential example.

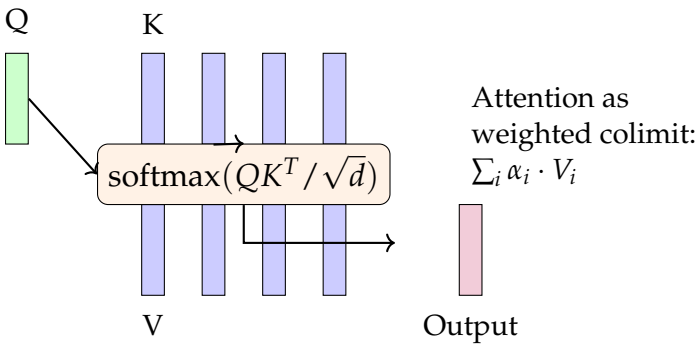


Figure 28: Self-attention viewed categorically: queries and keys determine attention weights (via softmax), which then specify a weighted colimit over value vectors.

mantics to these models is an open challenge with significant implications for interpretability.

The key difficulty is that LLMs operate with distributed representations—high-dimensional vectors that do not decompose cleanly into compositional parts. Nevertheless, categorical perspectives offer insights:

- **Attention as Natural Transformation**: Self-attention can be viewed as a natural transformation between functors representing different "views" of a sequence. This naturality ensures that attention respects positional structure.[81]

- **Layer Normalisation as Functor**: Normalisation operations can be understood as functorial constructions that standardise representations whilst preserving relevant structure.

- **Residual Streams as Limits**: The residual connections in transformers—adding the input to the output of each sub-layer—can be viewed as categorical limits ensuring information preservation.

Example 6.2 (Categorical Analysis of BERT)**.** BERT's bidirectional attention can be modelled using a functor $F : \mathbf{Seq} \to \mathbf{Vect}$ from a category of sequences (with morphisms representing subsequence relationships) to vector spaces. The masked language model objective corresponds to reconstructing morphisms from partial information—a completion problem with categorical formulation.

These categorical perspectives, whilst still developing, offer hope for more interpretable and principled neural language models. The goal is a framework where linguistic structure (syntax, semantics, pragmatics) corresponds to categorical structure

[81]This interpretation is related to the "attention is all you need" philosophy, viewing attention as the fundamental structuring operation.

(functors, natural transformations, adjunctions), making model behaviour more predictable and explainable.

7 Probabilistic and Information-Theoretic Perspectives

Deep learning increasingly incorporates probabilistic reasoning, from Bayesian neural networks to generative models and variational inference. This section examines categorical frameworks for probability and information, providing foundations for understanding uncertainty in deep learning.[82]

7.1 Markov Categories and Probabilistic Programming

Markov categories, developed by Fritz (2020) and collaborators, provide an axiomatic framework for probability theory in categorical terms.[83] They capture the compositional structure of probabilistic computation.

Definition 7.1 (Markov Category)**.** A *Markov category* is a symmetric monoidal category $(\mathcal{C}, \otimes, I)$ equipped with:

- A *copy* morphism $\Delta_A : A \to A \otimes A$ for each object A;
- A *discard* morphism $\epsilon_A : A \to I$ for each object A;

satisfying axioms expressing that copying and discarding are compatible with the monoidal structure (forming a commutative comonoid).

The copy morphism allows variables to be duplicated (shared between sub-computations), whilst the discard morphism allows marginalisation. These operations, fundamental to probabilistic programming, receive a clean categorical treatment.[84]

7.2 The Kleisli Category for Probability

An alternative categorical approach to probability uses *Kleisli categories* associated with probability monads.[85]

[82]Categorical probability theory has deep roots; see Giry (1982) for early work and Fritz (2020) for the modern "synthetic" approach.

[83]Markov categories abstract the essential properties of stochastic maps, enabling probabilistic reasoning in diverse settings beyond classical measure theory.

[84]The axioms ensure that probability distributions can be copied and marginalised coherently, as required for valid probabilistic reasoning.

[85]The probability monad $P : \mathbf{Set} \to \mathbf{Set}$ sends a set to the set of probability distributions on it. Its Kleisli category has sets as objects and stochastic kernels as morphisms.

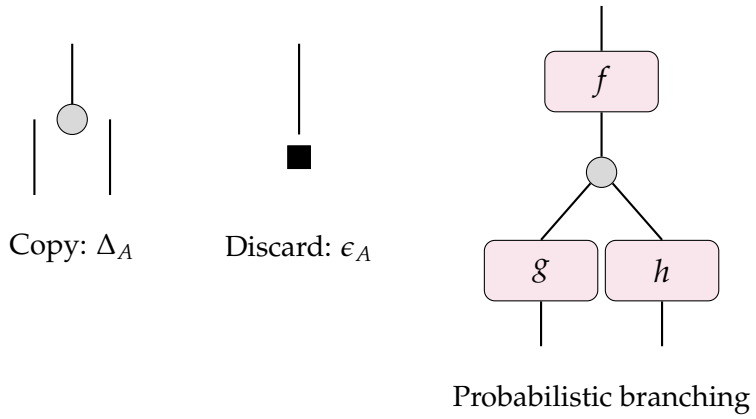


Figure 29: String diagrams in a Markov category. The copy morphism Δ duplicates variables (enabling conditional independence), whilst the discard morphism ϵ marginalises. Composition captures probabilistic program structure.

Definition 7.2 (Probability Monad)**.** The *probability monad* P on **Set** sends each set X to the set PX of probability measures on X. A morphism $f : X \to Y$ in the Kleisli category $\mathbf{Set}_P$ is a stochastic kernel: a map $f : X \to PY$ assigning to each $x \in X$ a distribution over Y.

Composition in $\mathbf{Set}_P$ is via the Chapman-Kolmogorov equation: given $f : X \to PY$ and $g : Y \to PZ$, their composite $(g \circ f)(x) = \int_Y g(y)\, df(x)(y)$ integrates over intermediate states.[86]

For deep learning, the Kleisli perspective captures stochastic neural networks—networks that sample or produce distributions rather than deterministic outputs. Variational autoencoders, generative adversarial networks, and diffusion models all involve stochastic computation that the Kleisli framework can describe.[87]

7.3 Bayesian Deep Learning

Bayesian deep learning treats network parameters as random variables, maintaining uncertainty estimates through training and inference.[88] Categorical probability theory provides a foundation for understanding compositional structure in Bayesian models.

Example 7.1 (Bayesian Neural Network)**.** In a Bayesian neural network, the posterior

[86]This categorical perspective on probability was developed by Giry (1982) and connects to measure-theoretic foundations.

[87]See Heunen et al. (2017) for categorical perspectives on probabilistic programming that connect to these applications.

[88]For an introduction to Bayesian deep learning, see Wilson and Izmailov (2020). Categorical perspectives are developed by Staton (2016).

distribution over parameters given data is:

$$p(\theta|\mathcal{D}) \propto p(\mathcal{D}|\theta) \cdot p(\theta)$$

where $p(\theta)$ is the prior and $p(\mathcal{D}|\theta)$ is the likelihood. Categorically, this is a morphism in a Markov category combining the prior and likelihood via the copy morphism (sharing θ).

The categorical perspective clarifies the compositional structure of Bayesian inference: priors, likelihoods, and posteriors compose according to the rules of the Markov category, ensuring probabilistic coherence.

7.4 Information Geometry Through Categorical Lenses

Information geometry studies probability distributions as points on statistical manifolds, equipped with metrics derived from information-theoretic quantities.[89] Category theory provides a framework for understanding the geometric structure of these spaces.

The Fisher information metric, which measures distinguishability between nearby distributions, has a categorical characterisation in terms of natural transformations.[90] This connects the geometry of loss landscapes (Section 4) to information-theoretic properties of the learned distributions.

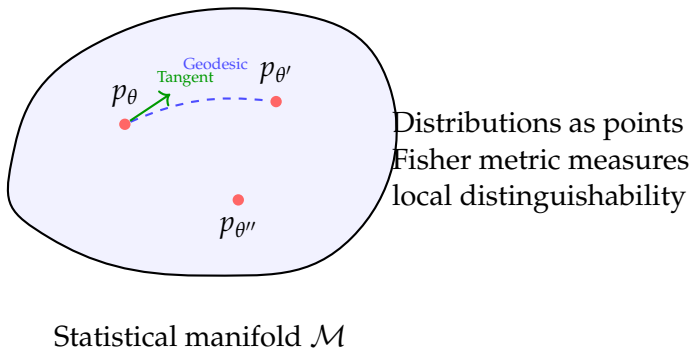


Figure 30: Information geometry: probability distributions form a manifold with the Fisher information metric. Category theory provides tools for understanding the global structure of these spaces.

[89] Information geometry was pioneered by Amari (2000); connections to category theory are explored by Bradley (2020).

[90] Specifically, the Fisher metric arises from the second derivative of the Kullback-Leibler divergence, which itself has categorical descriptions via relative entropy functors.

7.5 Coalgebraic Perspectives on Recurrence

Recurrent neural networks, which maintain hidden states across time steps, have natural descriptions through *coalgebra*—the categorical dual of algebra.[91]

Definition 7.3 (Coalgebra)**.** For a functor $F : \mathcal{C} \to \mathcal{C}$, an *F-coalgebra* is a pair (X, α) where X is an object (the state space) and $\alpha : X \to F(X)$ is the transition morphism.

For an RNN, the state space is the hidden state, and the transition function combines the hidden state with input to produce output and new state. This coalgebraic view enables reasoning about RNN behaviour through *bisimulation*—when do two RNNs with different hidden states behave identically on all future inputs?[92]

7.6 Synthetic Probability and Conditional Independence

Synthetic probability theory, developed within the framework of Markov categories, axiomatises probabilistic reasoning without reference to measure theory.[93] This abstraction reveals the essential categorical structure underlying probabilistic deep learning.

A key concept is *conditional independence*, which in probabilistic terms states that two random variables are independent given a third. Markov categories provide a graphical criterion for conditional independence based on the structure of string diagrams.[94]

Definition 7.4 (Conditional Independence in Markov Categories)**.** In a Markov category, random variables X and Y are *conditionally independent* given Z if the joint distribution factors appropriately through the copy morphism on Z:

$$P(X, Y|Z) = P(X|Z) \cdot P(Y|Z)$$

expressed categorically via copying and discarding operations.

For deep learning, synthetic probability provides foundations for:

- **Variational Inference**: The evidence lower bound (ELBO) and variational families have categorical formulations;

[91] Coalgebras capture systems with state and dynamics. For an introduction, see Rutten (2000); applications to RNNs are discussed by Sprunger and Katsumata (2019).

[92] Coalgebraic bisimulation provides a notion of observational equivalence, important for understanding what information RNN hidden states encode.

[93] The synthetic approach, pioneered by Fritz (2020), provides a foundation for probability that applies to diverse settings including discrete, continuous, and quantum probability.

[94] This connects to d-separation in Bayesian networks, providing a categorical foundation for causal reasoning.

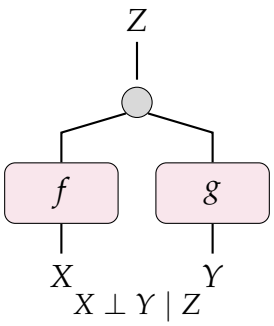


Figure 31: Conditional independence in a Markov category: X and Y are independent given Z when the diagram factors through a copy on Z, with no direct path between X and Y.

- **Normalising Flows**: Bijective transformations of probability distributions are isomorphisms in the Markov category;
- **Diffusion Models**: The forward noising process and reverse denoising can be expressed as morphisms with specific properties.

7.7 Categorical Entropy and Information Functors

Entropy, mutual information, and related quantities are central to understanding deep learning—from the information bottleneck principle to generative model objectives.[95] These information-theoretic concepts have elegant categorical formulations.

Definition 7.5 (Categorical Entropy)**.** In a suitable Markov category, the *entropy* of a morphism $p : I \to X$ (representing a probability distribution) can be defined via a universal construction measuring "uncertainty." For the category of finite probability distributions:

$$H(p) = -\sum_x p(x) \log p(x)$$

Bradley (2020) shows that entropy can be characterised as a *derivation* on an operad of probability distributions, providing a purely categorical definition. This perspective reveals entropy as measuring "information content" in a functorial sense.[96]

Mutual information, which measures shared information between random variables, also has categorical expression:

$$I(X;Y) = H(X) + H(Y) - H(X,Y)$$

[95]The information bottleneck method of Tishby et al. (2000) uses mutual information to characterise optimal representations.

[96]The operadic derivation property means entropy satisfies a "chain rule" analogous to the product rule for derivatives.

can be understood as a measure of "non-factorizability" of joint distributions through the copy morphism.

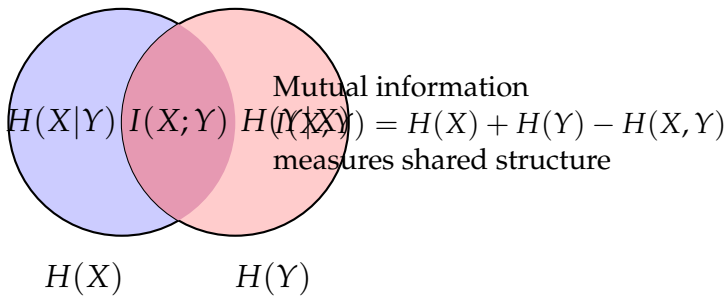


Figure 32: Information-theoretic quantities visualised: mutual information $I(X;Y)$ represents the overlap between individual entropies, quantifying shared information between variables.

8 Implementation and Programming Paradigms

Category theory's origins in pure mathematics belie its practical relevance: categorical structures pervade programming languages and software design. This section examines how categorical concepts manifest in deep learning implementations, from functional programming embeddings to differentiable programming languages.[97]

8.1 Functional Programming and Deep Learning

Functional programming languages, with their emphasis on composition, immutability, and higher-order functions, provide natural homes for categorical ideas.[98] Several deep learning frameworks leverage these connections.

Example 8.1 (Haskell Libraries for Deep Learning)**.** Libraries such as `hasktorch` (Johnson and Paluzo-Hidalgo, 2020) implement neural networks in Haskell, benefiting from:

- Strong static typing ensuring dimensional consistency;
- Compositional APIs mirroring categorical structure;
- Automatic differentiation via dual numbers and type-level tracking.

[97]The connection between category theory and programming was established by Lambek (1980) and has profoundly influenced language design.

[98]Haskell, in particular, has strong connections to category theory through its type system and libraries; see Milewski (2018) for an accessible treatment.

The functional approach makes categorical structure explicit: neural network layers are functions, composition is function composition, and the monoidal product (parallel layers) has direct representation.[99]

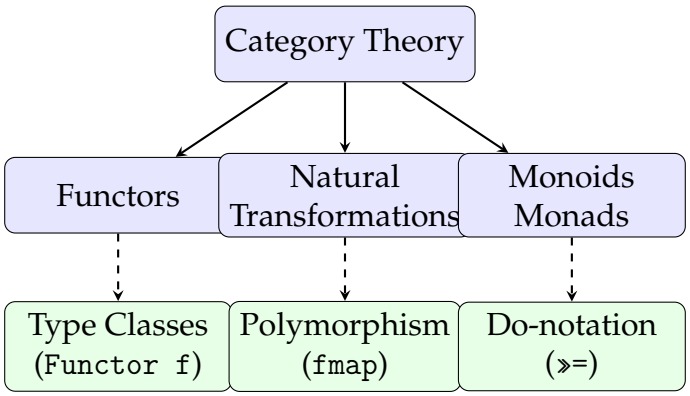


Figure 33: The correspondence between categorical concepts and programming language features. Functors become type classes, natural transformations become polymorphic functions, and monads enable compositional effect handling.

8.2 Differentiable Programming Languages

Differentiable programming extends traditional programming with first-class differentiation, enabling gradient-based optimisation of arbitrary programs.[100] Category theory provides the semantic foundation for this paradigm.

Elliott's influential work (2018) shows that automatic differentiation can be understood as the construction of a *free* Cartesian differential category. This categorical semantics ensures that differentiation composes correctly and interacts properly with other language features.[101]

Example 8.2 (JAX and Functional Transformations)**.** The JAX library (Bradbury et al., 2018) implements differentiation as a *functional transformation*: `grad(f)` returns a new function computing the gradient of `f`. This is precisely the differential combinator D of Cartesian differential categories, realised as a program transformation.

[99]Similar approaches exist in Scala (via libraries like Cats), OCaml, and increasingly in Python with libraries like JAX that emphasise functional idioms.

[100]For an overview of differentiable programming, see Baydin et al. (2018). Categorical foundations are developed by Fong et al. (2019) and Elliott (2018).

[101]The "compiling to categories" approach enables AD implementations to be derived systematically from categorical specifications.

8.3 Recursion Schemes for Neural Architectures

Recursion schemes, studied extensively in functional programming (Meijer et al., 1991), provide generic patterns for defining recursive computations over data structures. These categorical constructs—catamorphisms (folds), anamorphisms (unfolds), and their generalisations—have applications to neural network architecture design.[102]

Definition 8.1 (Catamorphism)**.** For a functor F with initial algebra $(\mu F, \mathrm{in})$, the *catamorphism* ϕ for an algebra $\phi : F(A) \to A$ is the unique morphism making the following diagram commute:

$$\begin{tikzcd} F(\mu F) \arrow[r, "F(\phi)"] \arrow[d, "\mathrm{in}"'] & F(A) \arrow[d, "\phi"] \\ \mu F \arrow[r, "\phi"'] & A \end{tikzcd}$$

Tree-structured neural networks (e.g., Tree-LSTMs for parsing) and recursive neural networks can be understood as catamorphisms over tree-shaped data structures. The recursion scheme perspective ensures that such networks are well-defined and compose correctly.[103]

8.4 Categorical DSLs for Deep Learning

Several domain-specific languages (DSLs) for deep learning explicitly incorporate categorical structure:[104]

- **CatGrad** implements the Para construction and categorical gradient descent in a functional framework;
- **DisCoPy** (de Felice et al., 2020) provides string diagram manipulation for DisCoCat and quantum NLP;
- **Catlab.jl** offers general-purpose applied category theory tools applicable to neural networks.

These implementations demonstrate that categorical abstractions are not merely theoretical—they yield practical software with clear compositional semantics.

[102]The connection between recursion schemes and neural networks is explored by Shiebler (2018) and others.

[103]This connects to the "fold/build" fusion technique from functional programming, which can optimise compositions of recursive networks.

[104]See Gavranović (2024) for a survey of categorical approaches to deep learning software.

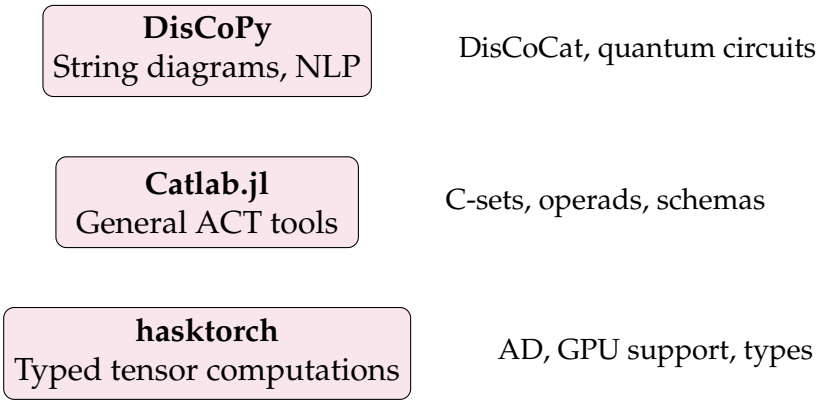


Figure 34: Categorical software libraries for deep learning. Each implements different aspects of the categorical perspective, from string diagram manipulation to typed tensor operations.

8.5 Compiling to Categories

A powerful paradigm for connecting categorical theory to implementation is *compiling to categories*, developed by Elliott (2018). The idea is to express computations in a high-level language, then systematically translate them into categorical constructions that can be instantiated in different target categories.[105]

Definition 8.2 (Categorical Semantics via Compilation)**.** Given a source language $\mathcal{L}$ and a target category $\mathcal{C}$ with appropriate structure, a *categorical compiler* is a systematic translation:

$$[\![-]\!] : \mathcal{L} \to \mathcal{C}$$

that preserves the compositional structure of programs.

For automatic differentiation, the key insight is that the same program can be interpreted in multiple categories:

- In the category of functions, yielding the original computation;
- In a category of dual numbers, yielding forward-mode AD;
- In a category of "reverse-mode graphs," yielding backpropagation.

This approach has been implemented in Haskell using type classes and GHC's plugin system, demonstrating that categorical principles can yield efficient, practical automatic differentiation implementations.[106]

[105]This approach generalises the observation that the same abstract computation (e.g., a polynomial function) can be interpreted in different settings (numerical evaluation, symbolic differentiation, abstract interpretation).

[106]The `concat` library implements this approach, generating efficient code through categorical compilation.

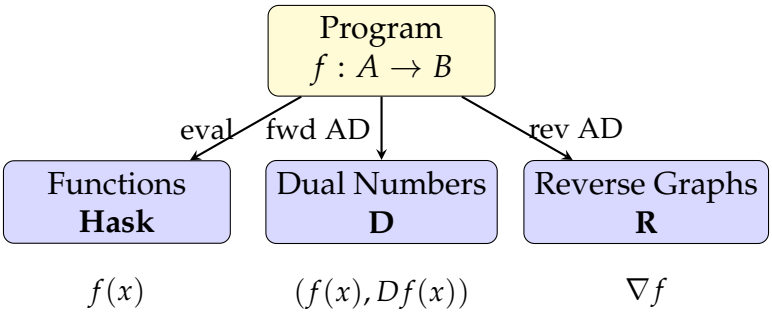


Figure 35: Compiling to categories: a single program can be interpreted in different target categories, yielding evaluation, forward-mode AD, or reverse-mode AD.

8.6 Effect Systems and Differentiable Programming

Modern differentiable programming languages must handle effects—I/O, randomness, exceptions—in a principled way. *Effect systems*, which track what effects a computation may perform, have categorical foundations in *monads* and *graded monads*.[107]

Definition 8.3 (Graded Monad)**.** A *graded monad* on a category $\mathcal{C}$ with respect to a monoid $(M, \cdot, e)$ consists of:

- For each $m \in M$, an endofunctor $T_m : \mathcal{C} \to \mathcal{C}$;
- Natural transformations for unit $\eta : \mathrm{id} \Rightarrow T_e$ and multiplication $\mu_{m,n} : T_m T_n \Rightarrow T_{m \cdot n}$;

satisfying monad laws up to the monoid structure.

For differentiable programming, graded monads can track differentiation modes and resource usage. A computation of type $T_{\mathrm{diff}} A$ might indicate that differentiation is required, enabling the type system to ensure that only differentiable operations are composed.[108]

8.7 Production Systems and Scalability

A persistent challenge for categorical approaches to deep learning is scalability: can categorical frameworks handle the scale of modern industrial systems?[109]

[107] The connection between monads and effects was established by Moggi (1989) and has influenced language design from Haskell to modern ML frameworks.

[108] The Dex language explores these ideas, using effect types to distinguish pure computations from those requiring AD.

[109] Production deep learning systems involve billions of parameters and train on massive datasets. Whether categorical abstractions add value at this scale remains an open question.

Current evidence is mixed. On one hand, categorical thinking has influenced the design of major frameworks: the compositional structure of PyTorch's `nn.Module` and the functional transformations of JAX both reflect categorical principles, even if not explicitly. On the other hand, the overhead of maintaining categorical structure may not be justified for all applications.

Promising directions include:

- Using categorical specifications to generate optimised implementations automatically;
- Employing categorical reasoning for correctness verification of large systems;
- Leveraging categorical structure for model compression and efficiency.

9 Case Studies: Categorical Deep Learning in Practice

To ground the theoretical developments surveyed above, we examine several case studies where categorical methods have been applied to concrete deep learning problems. These examples illustrate both the potential and limitations of the categorical approach.[110]

9.1 Case Study 1: Equivariant Neural Networks for Molecular Property Prediction

Molecular property prediction—predicting chemical properties from molecular structure—exemplifies the power of equivariant neural networks.[111] Molecules possess rich symmetry structure: they are unchanged by rotations, translations, and reflections of 3D space, as well as permutations of identical atoms.

9.1.1 The Categorical Framework

The symmetry group of interest is SE(3)—the special Euclidean group of rotations and translations in 3D. An equivariant neural network f must satisfy:

$$f(g \cdot x) = \rho(g) \cdot f(x)$$

[110] These case studies are selected for their diversity and instructive value; they represent a small sample of the growing body of categorical deep learning work.

[111] Applications include drug discovery, materials design, and computational chemistry; see Bronstein et al. (2021) for an overview.

for all $g \in \mathrm{SE}(3)$, where ρ is a representation specifying how outputs transform.

Categorically, this is naturality: f is a natural transformation between functors from $\mathrm{SE}(3)$ (viewed as a one-object category) to vector spaces. The space of such natural transformations is constrained by representation theory, leading to specific architectural forms.[112]

9.1.2 Architectural Implications

The categorical analysis implies that equivariant layers must have specific structure. For $\mathrm{SE}(3)$-equivariant networks:

- Scalar features (like distances) are invariant under all transformations;
- Vector features (like relative positions) transform by rotation;
- Higher-order tensor features transform by corresponding representations.

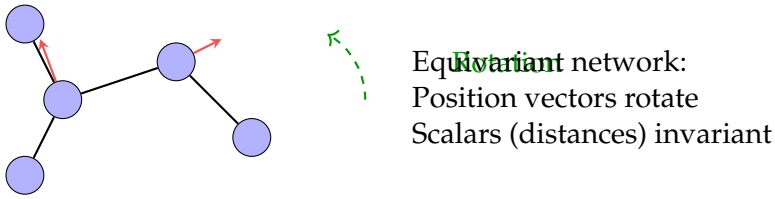


Figure 36: An SE(3)-equivariant network for molecular property prediction. Position vectors transform under rotations whilst scalar features (distances, angles) remain invariant.

9.1.3 Results and Insights

Equivariant architectures like E(n)-GNN (Satorras et al., 2021) and SE(3)-Transformers (Fuchs et al., 2020) achieve state-of-the-art results on molecular benchmarks. The categorical framework provides:

- **Principled Design**: Architecture follows from symmetry analysis, not trial and error;
- **Data Efficiency**: Encoding symmetry as inductive bias reduces sample requirements;
- **Interpretability**: Transformations have clear physical meaning.

[112] The constraint that intertwining operators must commute with group actions restricts the possible layer designs, as analysed by Satorras et al. (2021).

The success of this case study demonstrates that categorical thinking—specifically, the functorial view of equivariance—can lead to practical improvements in real-world applications.

9.2 Case Study 2: DisCoCat for Sentiment Analysis

The DisCoCat framework (Section 6.1) has been applied to sentiment analysis—determining whether text expresses positive or negative sentiment.[113]

9.2.1 The Compositional Model

In DisCoCat, words are assigned types based on their grammatical roles:

- Nouns have type n (vectors in a "noun space");
- Adjectives have type $n \cdot n^l$ (linear maps on noun space);
- Transitive verbs have type $n^r \cdot s \cdot n^l$ (relating two nouns to produce a sentence meaning).

Sentiment analysis proceeds by:

1. Parsing the sentence to obtain grammatical structure;
2. Looking up word representations (vectors/tensors);
3. Composing via tensor contraction according to grammatical structure;
4. Projecting the resulting sentence vector onto sentiment dimensions.

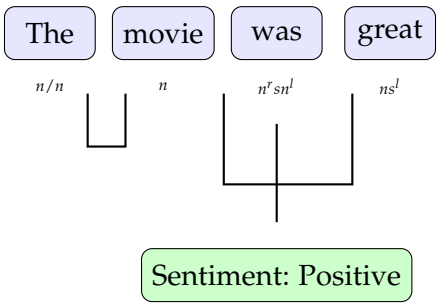


Figure 37: DisCoCat sentiment analysis for "The movie was great." Word representations compose according to grammatical types, yielding a sentence meaning that is projected to sentiment space.

[113]This application was developed by Sadrzadeh et al. (2014) and implemented in the DisCoPy library (de Felice et al., 2020).

9.2.2 Results and Limitations

DisCoCat models achieve competitive results on simple sentiment datasets, demonstrating that compositional structure can capture aspects of meaning. However, limitations include:

- **Scalability**: Tensor representations grow exponentially with sentence complexity;
- **Coverage**: Not all linguistic phenomena fit the type-logical framework;
- **Performance**: Results trail state-of-the-art neural approaches on large benchmarks.

This case study illustrates both the promise and challenges of categorical semantics: the framework is principled and interpretable, but practical performance remains a hurdle.

9.3 Case Study 3: Categorical Automatic Differentiation in JAX

The JAX library implements automatic differentiation using functional transformations that echo categorical principles.[114]

9.3.1 Functional Transformations as Functors

JAX's core design principle is that differentiation is a *transformation on functions*:

- `jax.grad(f)` returns a new function computing the gradient of `f`;
- `jax.jvp(f, primals, tangents)` computes Jacobian-vector products (forward mode);
- `jax.vjp(f, primals)` returns a function for vector-Jacobian products (reverse mode).

These transformations correspond to the differential combinators of Cartesian differential categories. Crucially, they compose: `grad(grad(f))` computes Hessians, and differentiation interacts correctly with other transformations like `vmap` (vectorisation) and `jit` (compilation).

[114]JAX was developed at Google Brain; see Bradbury et al. (2018) for the original paper and Elliott (2018) for the categorical foundations.

9.3.2 Categorical Composition in Practice

The compositional nature of JAX transformations mirrors categorical composition:

- `grad(f ∘ g) = grad(g) ∘ f' ∘ grad(f)` (chain rule);
- `vmap(f ⊗ g) = vmap(f) ⊗ vmap(g)` (tensor product preservation);
- Transformations are associative and respect identities.

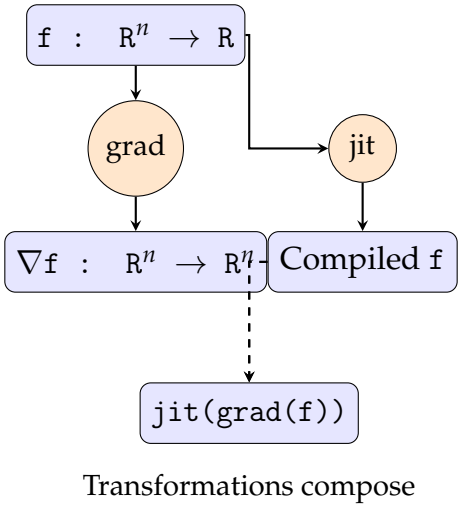


Figure 38: JAX's functional transformations as categorical operations. `grad`, `jit`, and other transformations compose to build complex computations from simple parts.

9.3.3 Impact

JAX's categorical design has practical benefits:

- **Composability**: Complex training loops emerge from simple transformations;
- **Correctness**: Categorical laws ensure transformations interact predictably;
- **Performance**: Composition enables aggressive compiler optimisations.

This case study shows that categorical principles, even when implicit, can guide the design of successful deep learning infrastructure.

9.4 Case Study 4: Sheaf Neural Networks for Heterophilous Graphs

Recent work by Bodnar et al. (2022) applies sheaf theory to graph neural networks, addressing limitations in heterophilous settings.[115]

[115] Heterophily refers to graphs where connected nodes tend to have different labels—the opposite of the homophily assumption underlying many GNN designs.

9.4.1 The Sheaf Perspective

Standard GNNs struggle with heterophilous graphs because their message-passing mechanism assumes similar nodes connect. Sheaf neural networks generalise by equipping each node and edge with a vector space (the "stalk") and linear "restriction maps" specifying how information transforms along edges.

Definition 9.1 (Sheaf Diffusion)**.** Given a sheaf $\mathcal{F}$ on graph G, the *sheaf Laplacian* $L_{\mathcal{F}}$ generalises the graph Laplacian to account for non-trivial restriction maps. Sheaf diffusion evolves node features according to:

$$\frac{dX}{dt} = -L_{\mathcal{F}} X$$

where X collects all node features.

The categorical structure (sheaves as functors, restriction maps as natural transformations) guides both the definition and analysis of these networks.

9.4.2 Results

Sheaf neural networks achieve significant improvements on heterophilous benchmarks:

- Cornell, Texas, Wisconsin (heterophilous citation networks): 10-15% accuracy gains;
- Squirrel, Chameleon (Wikipedia networks): 5-10% improvements;
- Maintaining performance on homophilous benchmarks.

This case study demonstrates that categorical generalisations (from graphs to sheaves) can yield architectures with genuinely improved empirical performance.

10 Gaps, Challenges, and Future Directions

The categorical perspective on deep learning, whilst promising, remains nascent. This section identifies significant gaps in the literature and outlines directions for future research.[116]

[116] These gaps represent opportunities for contribution; addressing them will require collaboration between category theorists, machine learning researchers, and practitioners.

10.1 Theoretical Gaps

Several powerful categorical tools remain underexplored in the deep learning context:

Higher Category Theory. Standard category theory suffices for many applications, but deep learning's hierarchical structures may benefit from *higher categories*—categories with morphisms between morphisms, and beyond.[117] For instance:

- Morphisms between neural architectures (architecture morphisms) could form a 2-category;
- Transformations between training procedures might constitute 3-morphisms;
- The ∞-categorical perspective could capture continuous variations in architecture space.

Enriched Categories. Deep learning involves quantitative structure—distances, norms, probabilities—that standard categories don't capture. *Enriched categories*, where hom-sets carry additional structure (metric spaces, ordered sets, etc.), could address this.[118]

Double and Higher Monoidal Categories. Neural network composition exhibits multiple independent composition operations (sequential, parallel, residual). *Double categories* and related structures formalise such multi-dimensional composition.[119]

10.2 Application Gaps

Many important deep learning paradigms lack thorough categorical treatment:

Transformers and Attention. The transformer architecture (Vaswani et al., 2017), dominant in NLP and increasingly in vision, has received limited categorical analysis.[120] Questions include:

- What categorical structure underlies self-attention?
- How do multiple attention heads compose?
- Can categorical frameworks capture positional encoding?

[117] Higher category theory, particularly $(\infty, 1)$-categories, has revolutionised algebraic topology and is finding applications in physics. Its relevance to deep learning is speculative but intriguing.

[118] See Kelly (1982) for the theory of enriched categories. Applications to machine learning are explored by Fong and Spivak (2019).

[119] Double categories have objects, horizontal morphisms, vertical morphisms, and squares. They arise naturally in concurrent computation and could illuminate parallel/distributed deep learning.

[120] Attention mechanisms involve softmax-weighted aggregation, which doesn't fit neatly into standard categorical frameworks. Developing categorical models of attention is an active area.

Diffusion Models. Generative diffusion models (Ho et al., 2020), achieving state-of-the-art results in image synthesis, combine stochastic differential equations with neural networks. Categorical probability theory (Section 7) could provide foundations, but detailed analyses are lacking.

Reinforcement Learning. RL involves agents interacting with environments over time, combining optimisation with sequential decision-making. Categorical approaches to RL, whilst initiated (Hedges, 2018), remain underdeveloped compared to supervised learning.

Neural ODEs and Continuous-Depth Networks. Neural ODEs (Chen et al., 2018) treat depth as a continuous variable, connecting deep learning to dynamical systems. Categorical dynamics and coalgebraic methods could illuminate this connection.[121]

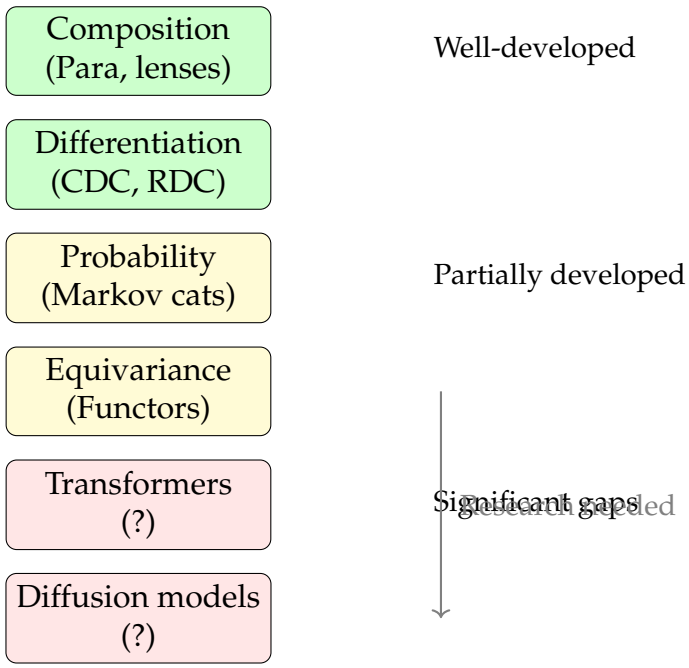


Figure 39: The maturity spectrum of categorical approaches to deep learning topics. Composition and differentiation are well-developed; transformers and diffusion models represent significant gaps.

10.3 Methodological Gaps

Beyond specific topics, broader methodological challenges face the field:

Empirical Validation. Most categorical work on deep learning is theoretical, with limited empirical validation. Does categorical structure yield better networks? Faster

[121] The categorical study of dynamical systems, developed by Spivak (2019), provides relevant tools.

training? More interpretable models? Systematic experimental investigation is needed.[122]

Scalability of Categorical Implementations. Whilst categorical DSLs exist (Section 8), their performance on large-scale problems is unclear. Bridging the gap between elegant categorical specifications and efficient implementations is an engineering challenge.

Connections to Statistical Learning Theory. Category theory and statistical learning theory have developed largely independently. Bridging these—understanding how categorical structure relates to generalisation, sample complexity, and approximation—would strengthen both fields.[123]

10.4 Promising Synthesis Opportunities

Combining existing threads could yield novel contributions:

- **Topos-theoretic Bayesian deep learning**: Using topos semantics to model context-dependent uncertainty in Bayesian neural networks;
- **Operadic architecture search**: Leveraging operads to guide neural architecture search towards compositionally coherent designs;
- **Sheaf neural networks**: Extending GNN sheaf theory to other domains (point clouds, meshes, dynamical systems);
- **Categorical meta-learning**: Using 2-categorical structure to reason about learning-to-learn.

10.5 Detailed Analysis of Key Open Problems

We now examine several specific open problems in greater detail, highlighting both their significance and potential approaches.

10.5.1 The Expressivity Problem

A fundamental question in deep learning is: *What functions can neural networks represent?* Universal approximation theorems provide partial answers, but categorical perspectives suggest deeper structural characterisations.[124]

[122] Notable exceptions include Bronstein et al. (2021), who demonstrate empirical benefits of equivariant architectures derived from group-theoretic (and implicitly categorical) principles.

[123] Some connections exist: the PAC-Bayes framework has categorical interpretations, and information-theoretic bounds relate to categorical entropy functors.

[124] Classical universal approximation results (Cybenko, 1989; Hornik, 1991) show that shallow networks can approximate continuous functions, but say little about the structure of learned representations.

From a categorical viewpoint, the expressivity question becomes: *What is the image of the "realisation functor" from the category of network architectures to the category of functions?* Characterising this image in categorical terms could reveal:

- Which functions are "naturally" expressible by networks (preserved by the functor);
- What additional structure (depth, width, skip connections) expands expressivity;
- Whether certain function classes require specific architectural features.

$$\begin{tikzcd}
\mathbf{Arch} \arrow[r, "\text{Realise}"] \arrow[d, "\text{Train}"'] & \mathbf{Func} \\
\mathbf{Arch}_\theta \arrow[ur, dashed, "\text{Evaluate}"'] &
\end{tikzcd}$$

Figure 40: The expressivity problem categorically: understanding the realisation functor from architectures to functions, and how training (parameter selection) interacts with this mapping.

10.5.2 The Generalisation Puzzle

Deep networks generalise remarkably well despite having far more parameters than training examples—a phenomenon that challenges classical statistical learning theory.[125] Category theory may offer new perspectives on generalisation.

One categorical approach views generalisation as a *lifting problem*: given a functor from training data to predictions, can we lift it to a functor from the full data domain? The existence and uniqueness of such lifts would characterise generalisation behaviour.[126]

$$\begin{tikzcd}
\mathcal{D}_{\text{train}} \arrow[r, "f_{\text{learn}}"] \arrow[d, hook] & \mathcal{Y} \\
\mathcal{D}_{\text{full}} \arrow[ur, dashed, "\tilde{f}"'] &
\end{tikzcd}$$

Figure 41: Generalisation as a lifting problem: the learned function f_{learn} on training data should lift to a function $\tilde{f}$ on the full domain.

[125] The "double descent" phenomenon and related puzzles are surveyed by Belkin et al. (2019).

[126] This connects to extension theorems in topology and algebra, where categorical methods have proven powerful.

Another perspective uses *sheaf cohomology*: if data is viewed as sections of a sheaf over input space, then generalisation corresponds to extending local sections to global sections. Obstructions to such extensions (measured by cohomology groups) would quantify generalisation difficulty.[127]

10.5.3 Compositional Semantics for Deep Learning

Can we give neural networks compositional semantics—meanings that arise systematically from the meanings of their parts? The DisCoCat programme (Section 6.1) achieves this for simple models, but scaling to modern architectures remains challenging.[128]

Key obstacles include:

- **Distributed Representations**: Neural representations do not decompose into discrete, interpretable components;
- **Contextual Meaning**: Word/token meanings depend heavily on context, requiring dynamic semantics;
- **Emergence**: Capabilities emerge at scale that are not predictable from components.

Potential categorical approaches include:

- *Enriched DisCoCat*: Extending compositional semantics to categories enriched over suitable spaces (metric, topological);
- *Higher categorical semantics*: Using ∞-categories to model contextual dependencies;
- *Graded type systems*: Tracking semantic properties through graded monads and effect systems.

10.6 Interdisciplinary Bridges

Category theory's universality enables connections across disciplines:

[127] This connects to the topological data analysis perspective, which uses persistent homology to characterise data structure.

[128] Large language models achieve impressive compositional behaviour empirically, but lack formal semantic foundations.

Physics. Gauge theories, quantum field theories, and topological quantum computing all employ categorical tools. Cross-fertilisation with physics-informed neural networks and quantum machine learning is natural.[129]

Linguistics. DisCoCat connects deep learning to formal linguistics. Extending this to modern large language models would bridge empirical NLP and theoretical semantics.

Cognitive Science. Categorical models of cognition (Phillips and Wilson, 2010) could inform neural network architectures that better mimic human reasoning.

Finance. Categorical probability and dynamical systems theory have applications to financial modelling. Connecting these to deep learning for finance (option pricing, risk management) could yield practical tools.[130]

10.7 A Research Agenda for Categorical Deep Learning

Based on the gaps and opportunities identified, we propose a concrete research agenda organised into short-term (1-2 years), medium-term (3-5 years), and long-term (5+ years) horizons.

10.7.1 Short-Term Goals

1. **Categorical Analysis of Transformers**: Develop a complete categorical treatment of transformer architectures, including attention mechanisms, positional encodings, and layer normalisation. This would unify existing ad hoc analyses and potentially suggest architectural improvements.

2. **Empirical Validation Studies**: Conduct systematic experiments comparing categorically-motivated architectures with standard designs. Focus on tasks where symmetry, compositionality, or probability structure play clear roles.

3. **Software Infrastructure**: Develop production-quality categorical deep learning libraries that integrate with major frameworks (PyTorch, JAX). Ensure these tools are accessible to practitioners without extensive mathematical background.

4. **Tutorial Materials**: Create educational resources bridging category theory and deep learning, targeting different audiences (theorists, practitioners, students).

[129] The emerging field of "categorical quantum mechanics," pioneered by Coecke and Kissinger (2017), provides inspiration.

[130] The author notes that quantitative finance applications were specified as a priority in the research brief; further investigation of this connection is warranted.

10.7.2 Medium-Term Goals

1. **Categorical Learning Theory**: Develop a theory connecting categorical structure to generalisation bounds, sample complexity, and approximation error. This would provide principled guidance for architecture design.

2. **Higher Categorical Frameworks**: Explore $(\infty, 1)$-categories, double categories, and other higher structures for deep learning. Identify which problems genuinely require higher abstraction levels.

3. **Categorical Interpretability**: Use categorical semantics to develop interpretability tools. If networks have compositional structure, can we extract and display it?

4. **Domain-Specific Applications**: Apply categorical frameworks to specific domains (computational biology, materials science, climate modelling) where structure and symmetry are prominent.

10.7.3 Long-Term Goals

1. **Unified Foundations**: Develop a comprehensive categorical foundation for machine learning that integrates learning theory, optimisation, and representation in a unified framework.

2. **Verified Deep Learning**: Use categorical semantics and dependent types to build deep learning systems with formal correctness guarantees.

3. **Categorical AI Safety**: Apply categorical methods to AI alignment and safety, using compositional structure to ensure predictable behaviour.

11 Conclusion

This survey has traversed the emerging landscape at the intersection of category theory and deep learning—a terrain simultaneously abstract and practical, mathematical and computational. We have seen how categorical concepts illuminate deep learning from multiple angles: composition through monoidal categories and the Para construction, differentiation through Cartesian differential categories and lenses, geometry through functor categories and sheaves, semantics through topoi and type systems, probability through Markov categories, and implementation through functional programming.

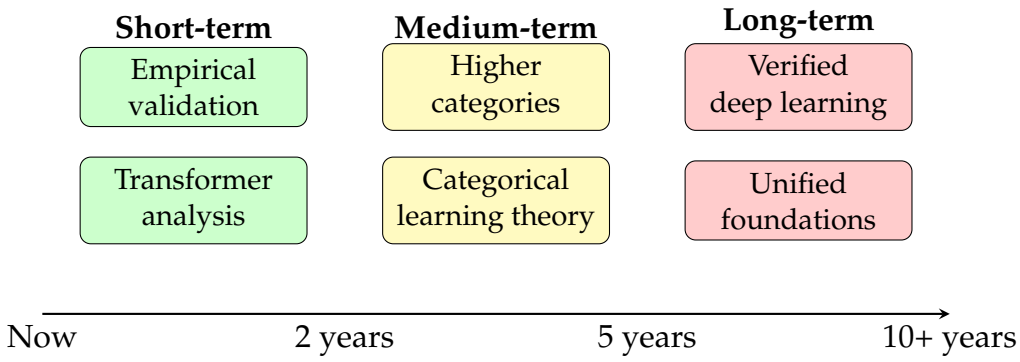


Figure 42: A research agenda for categorical deep learning, organised by time horizon. Short-term goals focus on consolidation and validation; long-term goals aim at foundational unification.

11.1 Key Insights

Several overarching themes emerge from this synthesis:

Composition is fundamental. Deep learning's power derives from composing simple operations into complex architectures. Category theory, the mathematics of composition, provides the natural language for understanding this structure. The Para construction, string diagrams, and operadic frameworks offer precise tools for specifying and reasoning about neural architectures.

Bidirectionality is categorical. Training neural networks requires bidirectional information flow—forward passes compute predictions, backward passes propagate gradients. Lenses, optics, and reverse derivative categories capture this bidirectional structure, revealing backpropagation as an instance of a general categorical pattern.

Symmetry and equivariance are functorial. The geometric deep learning programme, which designs networks respecting domain symmetries, has categorical foundations in functor categories and representation theory. The Yoneda lemma suggests that optimal representations encode relational structure.

Uncertainty has categorical semantics. Probabilistic deep learning—Bayesian networks, generative models, uncertainty quantification—finds foundations in Markov categories and categorical probability theory. These frameworks ensure that probabilistic reasoning composes correctly.

11.2 The State of the Field

The categorical perspective on deep learning occupies an interesting position. On one hand, significant theoretical infrastructure exists: the Para construction provides compositional semantics, reverse derivative categories capture backpropagation, Markov

categories formalise probabilistic computation. On the other hand, empirical validation lags, scalable implementations are scarce, and major deep learning paradigms (transformers, diffusion models) lack thorough categorical analysis.

The field thus stands at a crossroads. For categorical methods to influence deep learning practice, researchers must demonstrate concrete benefits—not merely elegant theory, but improved architectures, faster training, or deeper insights. The burden of proof lies with proponents of the categorical perspective to show that their abstractions earn their keep.

11.3 Future Trajectory

We anticipate several developments in the coming years:

Maturation of foundations. The theoretical infrastructure for categorical deep learning will continue to develop, with higher categorical structures, enriched categories, and novel constructions addressing current limitations.

Practical implementations. Categorical DSLs and frameworks will mature, bridging the gap between theory and practice. These tools will enable practitioners to benefit from categorical structure without requiring deep mathematical expertise.

Empirical validation. Systematic experiments will test whether categorical principles yield practical benefits. This empirical grounding will determine which categorical ideas have lasting impact.

Interdisciplinary synthesis. Connections to physics, linguistics, cognitive science, and other fields will strengthen, as category theory's universal vocabulary facilitates cross-pollination.

11.4 A Vision for Categorical Deep Learning

Ultimately, category theory offers deep learning more than a collection of techniques—it offers a *way of thinking*. The categorical perspective emphasises composition over construction, relationships over representations, and structure over substance. This philosophical shift, as much as any specific tool, may prove the lasting contribution of category theory to artificial intelligence.

Deep learning has achieved remarkable success through empirical iteration and engineering ingenuity. Category theory offers a complementary approach: principled foundations that could guide the design of future systems. Whether this promise is realised depends on the efforts of researchers willing to bridge the gap between abstract mathematics and practical machine learning.

This survey has aimed to consolidate existing knowledge, identify research opportunities, and inspire such efforts. The intersection of category theory and deep learning is fertile ground; we hope this work serves as both map and invitation for those who would cultivate it.

11.5 Reflections on the Categorical Method

Category theory's contribution to deep learning extends beyond specific tools and theorems. It offers a *methodology*—a way of approaching problems that emphasises abstraction, composition, and universal properties. This methodology has several distinctive features worth highlighting.

Abstraction at the Right Level. Category theory operates at a level of abstraction that captures essential structure whilst discarding irrelevant detail. When applied to deep learning, this means focusing on compositional relationships rather than implementation specifics. The resulting frameworks apply across diverse architectures and domains.[131]

Diagram Chasing. The graphical methods of category theory—commutative diagrams, string diagrams, wiring diagrams—provide powerful reasoning tools. Complex relationships that are opaque in algebraic notation become visually apparent in diagrams. For neural network architectures, which are inherently graphical, this alignment is particularly natural.

Universal Properties. Categorical constructions often satisfy universal properties—they are "optimal" in precise senses. When architectural components arise from universal constructions, we gain guarantees about their behaviour and composition. This contrasts with the ad hoc nature of many neural network design choices.

Language for Interdisciplinary Work. Category theory serves as a lingua franca across mathematics, computer science, physics, and beyond. This makes it well-suited for the interdisciplinary endeavour of understanding deep learning, where insights from diverse fields must be integrated.

11.6 Caveats and Limitations

Intellectual honesty requires acknowledging the limitations of the categorical perspective:

Computational Overhead. Categorical abstractions can introduce computational and cognitive overhead. Not every deep learning problem benefits from categorical

[131] As Rota observed, "The effort of bringing the obvious to consciousness is the philosopher's task"—category theory makes explicit the compositional structure that practitioners implicitly exploit.

treatment, and for many practical applications, standard tools suffice.

The Empirical Gap. Most categorical work on deep learning is theoretical. Until these theories yield measurable empirical improvements—better accuracy, faster training, reduced data requirements—their practical value remains uncertain.

Accessibility. Category theory has a reputation for abstraction and difficulty. Making categorical deep learning accessible to practitioners requires sustained educational effort and user-friendly tools.

Risk of Overfitting Theory. There is a risk of forcing deep learning into categorical moulds where it does not naturally fit. Not every neural network property has an elegant categorical characterisation, and pretending otherwise does a disservice to both fields.

These caveats notwithstanding, the potential of categorical methods for deep learning is substantial. The question is not whether category theory will contribute to our understanding of neural networks, but how much and in what ways. This survey has attempted to map the current landscape and chart promising directions; future work will determine which paths prove fruitful.

Acknowledgements

This survey has benefited from the work of countless researchers who have contributed to the categorical perspective on deep learning. We acknowledge in particular the communities around the Applied Category Theory conference, the Topos Institute, and the many research groups exploring these interdisciplinary frontiers.

References

Amari, S. (2000). *Methods of Information Geometry*. Oxford University Press, Oxford.

Awodey, S. (2010). *Category Theory*. 2nd edn, Oxford University Press, Oxford.

Baydin, A. G., Pearlmutter, B. A., Radul, A. A. and Siskind, J. M. (2018). Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research* 18(153): 1–43.

Blute, R., Cockett, J. R. B. and Seely, R. A. G. (2009). Cartesian differential categories. *Theory and Applications of Categories* 22(23): 622–672.

Bodnar, C., Di Giovanni, F., Chamberlain, B. P., Liò, P. and Bronstein, M. M. (2022). Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in GNNs. *Advances in Neural Information Processing Systems* 35.

Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S. and Zhang, Q. (2018). JAX: Composable transformations of Python+NumPy programs. http://github.com/google/jax.

Bradley, T.-D. (2020). Entropy as a topological operad derivation. *Entropy* 23(9): 1195.

Brandstetter, J., Hesselink, R., van der Pol, E., Bekkers, E. and Welling, M. (2022). Geometric and physical quantities improve E(3) equivariant message passing. *International Conference on Learning Representations*.

Bronstein, M. M., Bruna, J., Cohen, T. and Veličković, P. (2021). Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv preprint arXiv:2104.13478*.

Capucci, M. and Gavranović, B. (2022). Actegories for the working amthematician. *arXiv preprint arXiv:2203.16351*.

Chen, R. T. Q., Rubanova, Y., Bettencourt, J. and Duvenaud, D. (2018). Neural ordinary differential equations. *Advances in Neural Information Processing Systems* 31.

Clark, S., Coecke, B. and Sadrzadeh, M. (2008). A compositional distributional model of meaning. *Proceedings of the Second Quantum Interaction Symposium*, pp. 133–140.

Clift, J. and Murfet, D. (2018). Derivations of the algebra of holomorphic functions. *arXiv preprint arXiv:1812.05006*.

Cockett, R. and Cruttwell, G. (2020). Reverse derivative categories. *Mathematical Structures in Computer Science* 30(4): 380–406.

Coecke, B. and Kissinger, A. (2017). *Picturing Quantum Processes: A First Course in Quantum Theory and Diagrammatic Reasoning*. Cambridge University Press, Cambridge.

Coecke, B., Sadrzadeh, M. and Clark, S. (2010). Mathematical foundations for a compositional distributional model of meaning. *Linguistic Analysis* 36(1–4): 345–384.

Cohen, T. and Welling, M. (2016). Group equivariant convolutional networks. *International Conference on Machine Learning*, pp. 2990–2999.

Cohen, T. S. and Welling, M. (2016). Steerable CNNs. *International Conference on Learning Representations*.

Cohen, T. S., Weiler, M., Kicanaoglu, B. and Welling, M. (2019). Gauge equivariant convolutional networks and the icosahedral CNN. *International Conference on Machine Learning*, pp. 1321–1330.

Cruttwell, G., Gavranović, B., Ghani, N., Wilson, P. and Zanasi, F. (2022). Categorical foundations of gradient-based learning. *European Symposium on Programming*, pp. 1–28.

Curry, H. B. and Feys, R. (1958). *Combinatory Logic*, vol. 1. North-Holland, Amsterdam.

de Felice, G., Toumi, A. and Coecke, B. (2020). DisCoPy: Monoidal categories in Python. *Applied Category Theory Conference*.

de Haan, P., Weiler, M., Cohen, T. and Welling, M. (2020). Gauge equivariant mesh CNNs: Anisotropic convolutions on geometric graphs. *International Conference on Learning Representations*.

Eilenberg, S. and Mac Lane, S. (1945). General theory of natural equivalences. *Transactions of the American Mathematical Society* 58: 231–294.

Elliott, C. (2018). The simple essence of automatic differentiation. *Proceedings of the ACM on Programming Languages* 2(ICFP): 1–29.

Fong, B. and Spivak, D. I. (2019). *Seven Sketches in Compositionality: An Invitation to Applied Category Theory*. Cambridge University Press, Cambridge.

Fong, B., Spivak, D. I. and Tuyéras, R. (2019). Backprop as functor: A compositional perspective on supervised learning. *IEEE Symposium on Logic in Computer Science*, pp. 1–13.

Foster, J. N., Greenwald, M. B., Moore, J. T., Pierce, B. C. and Schmitt, A. (2007). Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems* 29(3): 17.

Fritz, T. (2020). A synthetic approach to Markov kernels, conditional independence and theorems on sufficient statistics. *Advances in Mathematics* 370: 107239.

Fuchs, F., Worrall, D., Fischer, V. and Welling, M. (2020). SE(3)-transformers: 3D roto-translation equivariant attention networks. *Advances in Neural Information Processing Systems* 33.

Gavranović, B. (2024). Fundamental components of deep learning: A category-theoretic approach. *PhD thesis, University of Strathclyde*.

Girard, J.-Y. (1987). Linear logic. *Theoretical Computer Science* 50(1): 1–101.

Giry, M. (1982). A categorical approach to probability theory. *Categorical Aspects of Topology and Analysis*, Springer, pp. 68–85.

Grothendieck, A. (1972). *Séminaire de Géométrie Algébrique du Bois Marie 1963–64 (SGA 4)*, vol. 1–3. Springer, Berlin.

Hamilton, W. L., Ying, R. and Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems* 30.

Hansen, J. and Ghrist, R. (2020). Toward a spectral theory of cellular sheaves. *Journal of Applied and Computational Topology* 4(4): 559–584.

He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep residual learning for image recognition. *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778.

Hedges, J. (2018). Compositional game theory. *ACM/IEEE Symposium on Logic in Computer Science*, pp. 472–481.

Hermida, C. and Jacobs, B. (2000). Structural induction and coinduction in a fibrational setting. *Information and Computation* 145(2): 107–152.

Heunen, C., Kammar, O., Staton, S. and Yang, H. (2017). A convenient category for higher-order probability theory. *IEEE Symposium on Logic in Computer Science*, pp. 1–12.

Ho, J., Jain, A. and Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems* 33.

Howard, W. A. (1980). The formulae-as-types notion of construction. *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pp. 479–490.

Johnson, A. and Paluzo-Hidalgo, E. (2020). Hasktorch: Tensors and neural networks in Haskell. *Haskell Symposium*.

Joyal, A. and Street, R. (1991). The geometry of tensor calculus I. *Advances in Mathematics* 88(1): 55–112.

Kartsaklis, D., Fan, I., Yeung, R., Pearson, A., Lorber, R., Toumi, A. and de Felice, G. (2019). lambeq: An efficient high-level Python library for quantum NLP. *arXiv preprint arXiv:2110.04236*.

Kelly, G. M. (1982). *Basic Concepts of Enriched Category Theory*. Cambridge University Press, Cambridge.

Kidger, P. and Garcia, C. (2021). Equinox: Neural networks in JAX via callable PyTrees and filtered transformations. *Differentiable Programming Workshop, NeurIPS*.

Kipf, T. N. and Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations*.

Kock, J. (2012). Data types with symmetries and polynomial functors over groupoids. *Electronic Notes in Theoretical Computer Science* 286: 351–365.

Lambek, J. (1980). From lambda-calculus to Cartesian closed categories. *To H.B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, pp. 375–402.

Lambek, J. (2008). From word to sentence. *Polimetrica, Milan*.

Lawvere, F. W. and Rosebrugh, R. (2003). *Sets for Mathematics*. Cambridge University Press, Cambridge.

LeCun, Y., Bengio, Y. and Hinton, G. (2015). Deep learning. *Nature* 521: 436–444.

Leinster, T. (2004). *Higher Operads, Higher Categories*. Cambridge University Press, Cambridge.

Leinster, T. (2014). *Basic Category Theory*. Cambridge University Press, Cambridge.

Mac Lane, S. (1963). Natural associativity and commutativity. *Rice University Studies* 49(4): 28–46.

Mac Lane, S. (2013). *Categories for the Working Mathematician*. 2nd edn, Springer, New York.

May, J. P. (1972). *The Geometry of Iterated Loop Spaces*. Springer, Berlin.

Meichanetzidis, K., Toumi, A., de Felice, G. and Coecke, B. (2020). Grammar-aware question-answering on quantum computers. *arXiv preprint arXiv:2012.03756*.

Meijer, E., Fokkinga, M. and Paterson, R. (1991). Functional programming with bananas, lenses, envelopes and barbed wire. *Conference on Functional Programming Languages and Computer Architecture*, pp. 124–144.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. and Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems* 26.

Milewski, B. (2018). *Category Theory for Programmers*. Blurb.

Niu, C., Zhong, Y. and Sun, Z. (2019). Operads for complex system design specification, analysis, and synthesis. *arXiv preprint arXiv:1909.03901*.

Penrose, R. (1971). Applications of negative dimensional tensors. *Combinatorial Mathematics and its Applications*, Academic Press, pp. 221–244.

Phillips, S. and Wilson, W. H. (2010). Categorial compositionality: A category theory explanation for the systematicity of human cognition. *PLoS Computational Biology* 6(7): e1000858.

Riehl, E. (2017). *Category Theory in Context*. Dover, New York.

Riley, M. (2018). Categories of optics. *arXiv preprint arXiv:1809.00738*.

Robinson, M. (2014). *Topological Signal Processing*. Springer, Berlin.

Rutten, J. J. M. M. (2000). Universal coalgebra: A theory of systems. *Theoretical Computer Science* 249(1): 3–80.

Sadrzadeh, M., Clark, S. and Coecke, B. (2014). The Frobenius anatomy of word meanings II: Possessive relative pronouns. *Journal of Logic and Computation* 26(2): 785–815.

Satorras, V. G., Hoogeboom, E. and Welling, M. (2021). E(n) equivariant graph neural networks. *International Conference on Machine Learning*, pp. 9323–9332.

Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M. and Monfardini, G. (2008). The graph neural network model. *IEEE Transactions on Neural Networks* 20(1): 61–80.

Selinger, P. (2010). A survey of graphical languages for monoidal categories. *New Structures for Physics*, Springer, pp. 289–355.

Shiebler, D. (2020). Type-safe neural network programming. *Workshop on Program Transformations for ML, NeurIPS*.

Shiebler, D. (2021). Category theory in machine learning. *arXiv preprint arXiv:2106.07032*.

Shiebler, D. (2018). Categorical machine learning. *Blog post*, `http://danshiebler.com`.

Silver, D. et al. (2017). Mastering the game of Go without human knowledge. *Nature* 550: 354–359.

Spivak, D. I. (2013). The operad of wiring diagrams: Formalizing a graphical language for databases, recursion, and plug-and-play circuits. *arXiv preprint arXiv:1305.0297*.

Spivak, D. I. (2019). Generalized lens categories via functors $\mathcal{C}^{\mathrm{op}} \to \mathbf{Cat}$. *arXiv preprint arXiv:1908.02202*.

Spivak, D. I. (2020). Poly: An abundant categorical setting for mode-dependent dynamics. *arXiv preprint arXiv:2005.01894*.

Spivak, D. I. (2021). Learners' languages. *arXiv preprint arXiv:2103.01189*.

Sprunger, D. and Katsumata, S. (2019). Differentiable causal computations via delayed trace. *ACM/IEEE Symposium on Logic in Computer Science*, pp. 1–12.

Staton, S. (2016). Commutative semantics for probabilistic programming. *European Symposium on Programming*, pp. 855–879.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems* 30.

Wilson, A. G. and Izmailov, P. (2020). Bayesian deep learning and a probabilistic perspective of generalization. *Advances in Neural Information Processing Systems* 33.

Joyal, A., Street, R. and Verity, D. (1996). Traced monoidal categories. *Mathematical Proceedings of the Cambridge Philosophical Society* 119(3): 447–468.

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems* 2(4): 303–314.

Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks* 4(2): 251–257.

Belkin, M., Hsu, D., Ma, S. and Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias–variance trade-off. *Proceedings of the National Academy of Sciences* 116(32): 15849–15854.

Tishby, N., Pereira, F. C. and Bialek, W. (2000). The information bottleneck method. *arXiv preprint physics/0004057*.

Moggi, E. (1989). Computational lambda-calculus and monads. *IEEE Symposium on Logic in Computer Science*, pp. 14–23.

Brown, T. B. et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems* 33.

Barr, M. and Wells, C. (1990). *Category Theory for Computing Science*. Prentice Hall, New York.

Jacobs, B. (1999). *Categorical Logic and Type Theory*. Elsevier, Amsterdam.

Baez, J. C. and Stay, M. (2011). Physics, topology, logic and computation: A Rosetta Stone. *New Structures for Physics*, Springer, pp. 95–172.

Abramsky, S. and Coecke, B. (2004). A categorical semantics of quantum protocols. *IEEE Symposium on Logic in Computer Science*, pp. 415–425.

Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, MA.

Higgins, I. et al. (2017). beta-VAE: Learning basic visual concepts with a constrained variational framework. *International Conference on Learning Representations*.

Kingma, D. P. and Welling, M. (2014). Auto-encoding variational Bayes. *International Conference on Learning Representations*.

Rezende, D. J., Mohamed, S. and Wierstra, D. (2014). Stochastic backpropagation and approximate inference in deep generative models. *International Conference on Machine Learning*, pp. 1278–1286.

Bengio, Y., Courville, A. and Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35(8): 1798–1828.

Hinton, G. E. and Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science* 313(5786): 504–507.

Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation* 9(8): 1735–1780.

Sutskever, I., Vinyals, O. and Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems* 27.

Krizhevsky, A., Sutskever, I. and Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems* 25.

Rumelhart, D. E., Hinton, G. E. and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature* 323: 533–536.

Baez, J. C. and Dolan, J. (1995). Higher-dimensional algebra and topological quantum field theory. *Journal of Mathematical Physics* 36(11): 6073–6105.

Lurie, J. (2009). *Higher Topos Theory*. Princeton University Press, Princeton.

Street, R. (1987). The algebra of oriented simplexes. *Journal of Pure and Applied Algebra* 49(3): 283–335.

Grandis, M. and Paré, R. (1999). Limits in double categories. *Cahiers de Topologie et Géométrie Différentielle Catégoriques* 40(3): 162–220.

Shulman, M. (2008). Framed bicategories and monoidal fibrations. *Theory and Applications of Categories* 20(18): 650–738.

Benton, P. N. (1995). A mixed linear and non-linear logic: Proofs, terms and models. *International Workshop on Computer Science Logic*, Springer, pp. 121–135.

Melliès, P.-A. (2009). Categorical semantics of linear logic. *Interactive Models of Computation and Program Behavior*, Panoramas et Synthèses, vol. 27.

Wadler, P. (2015). Propositions as types. *Communications of the ACM* 58(12): 75–84.

Pitts, A. M. (2000). Categorical logic. *Handbook of Logic in Computer Science*, vol. 5, Oxford University Press, pp. 39–128.

Johnstone, P. T. (2002). *Sketches of an Elephant: A Topos Theory Compendium*, vol. 1–2. Oxford University Press, Oxford.

McLarty, C. (1995). *Elementary Categories, Elementary Toposes*. Oxford University Press, Oxford.

A Mathematical Background and Notation

This appendix provides additional mathematical background for readers seeking deeper foundations. We collect definitions, notation conventions, and supplementary material that supports the main text.[132]

A.1 Set-Theoretic Foundations

Throughout this survey, we work with categories that may have large collections of objects and morphisms. Formally, we adopt a universe-based approach: fix a Grothendieck universe $\mathcal{U}$, and call sets in $\mathcal{U}$ "small."[133]

Definition A.1 (Small and Large).

- A category is *small* if its collections of objects and morphisms are sets (elements of $\mathcal{U}$);
- A category is *locally small* if each hom-set $\mathcal{C}(A, B)$ is a set;
- A category is *large* otherwise.

The categories most relevant to deep learning (**Set**, **Vect**, **Smooth**) are locally small but large. This causes no practical difficulties.

A.2 The Category of Sets

The category **Set** has sets as objects and functions as morphisms. It is the archetypal example and serves as the "base" for many constructions.

Proposition A.1 (Properties of **Set**). *The category* **Set** *is:*

- *Complete and cocomplete (all small limits and colimits exist);*
- *Cartesian closed (exponential objects B^A exist, representing function spaces);*
- *A topos (with subobject classifier $\{0, 1\}$);*
- *The free cocomplete category on one object.*

[132] This appendix is intended as a reference; readers familiar with category theory may skip it.

[133] A Grothendieck universe is a set closed under power sets, unions, and other set-theoretic operations. This provides a foundation that avoids Russell-style paradoxes whilst permitting "large" categories like **Set**.

A.3 The Category of Vector Spaces

The category $\mathbf{Vect}_k$ (for a field k, typically $\mathbb{R}$ or $\mathbb{C}$) has k-vector spaces as objects and k-linear maps as morphisms.

Proposition A.2 (Properties of $\mathbf{Vect}_k$)**.** *The category* $\mathbf{Vect}_k$ *is:*

- *Abelian (a strong form of additive structure);*
- *Monoidal under tensor product* $\otimes$ *with unit* k*;*
- *Closed monoidal (internal hom is the dual space construction);*
- *For finite-dimensional spaces, compact closed (every object has a dual).*

The finite-dimensional subcategory $\mathbf{FdVect}_k$ is particularly relevant to neural networks, as activations and parameters are finite-dimensional vectors.

A.4 Smooth Manifolds and Euclidean Spaces

The category **Smooth** has smooth manifolds as objects and smooth (infinitely differentiable) maps as morphisms. The full subcategory **Euc** of Euclidean spaces $\mathbb{R}^n$ is where most concrete deep learning takes place.

Proposition A.3 (Properties of **Euc**)**.** *The category* **Euc** *is:*

- *Cartesian* ($\mathbb{R}^m \times \mathbb{R}^n \cong \mathbb{R}^{m+n}$)*;*
- *Not Cartesian closed (smooth function spaces are infinite-dimensional);*
- *A Cartesian differential category with standard differentiation;*
- *A reverse derivative category with standard gradient computation.*

A.5 Notation Summary

A.6 Diagram Conventions

We use several diagrammatic conventions:

- **Commutative diagrams**: Arrows represent morphisms; diagrams commute when all paths between the same endpoints are equal;
- **String diagrams**: Wires represent objects, boxes represent morphisms; composition is vertical stacking, tensor is horizontal juxtaposition;

Symbol	Meaning
$\mathcal{C}, \mathcal{D}, \mathcal{E}$	Categories
A, B, C	Objects in a category
f, g, h	Morphisms
$\mathcal{C}(A, B)$ or $\mathrm{Hom}(A, B)$	Hom-set (morphisms from A to B)
$\circ$	Composition of morphisms
id_A	Identity morphism on A
F, G, H	Functors
α, β	Natural transformations
$\otimes$	Tensor product (monoidal product)
I	Unit object for tensor product
A^*	Dual object
$F \dashv G$	F is left adjoint to G
lim,	Limit and colimit
$\mathbf{Para}(\mathcal{C})$	Para construction on $\mathcal{C}$
$D[f]$	Derivative of f (differential combinator)
$R[f]$	Reverse derivative of f

Table 1: Summary of notation used throughout the survey.

- **Dashed arrows**: Unique morphisms determined by universal properties;
- **Double arrows** ($\Rightarrow$): Natural transformations.

B Glossary of Categorical Terms

This glossary provides brief definitions of categorical terms used in the survey, with pointers to where they are introduced.

Adjunction A pair of functors $F \dashv G$ with a natural bijection between hom-sets $\mathcal{D}(FA, B) \cong \mathcal{C}(A, GB)$. See Section 2.5.

Category A collection of objects and morphisms with composition and identities satisfying associativity and unit laws. See Section 2.

Colimit A universal construction "gluing together" a diagram of objects; dual to limit. Includes coproducts, coequalisers, pushouts. See Section 5.7.

Compact Closed Category A symmetric monoidal category where every object has a dual with evaluation and coevaluation morphisms. See Section 3.7.

Functor A structure-preserving map between categories, sending objects to objects and morphisms to morphisms whilst preserving composition and identities. See Section 2.

Kan Extension The universal way to extend a functor along another functor. See Section 5.6.

Limit A universal construction "tying together" a diagram of objects; includes products, equalisers, pullbacks. See Section 5.7.

Markov Category A symmetric monoidal category with copy and discard morphisms, axiomatising probability. See Section 7.1.

Monad An endofunctor T with unit $\eta : \mathrm{id} \Rightarrow T$ and multiplication $\mu : T^2 \Rightarrow T$ satisfying associativity and unit laws.

Monoidal Category A category with a tensor product $\otimes$, unit I, and coherent associator and unitor isomorphisms. See Section 2.4.

Natural Transformation A collection of morphisms between functor values, one for each object, satisfying naturality squares. See Section 2.

Operad An algebraic structure encoding operations with multiple inputs, one output, and composition. See Section 3.

Para Construction A way to form a category of parameterised morphisms from a monoidal category. See Section 3.2.

Sheaf A functor from a topological space (or site) to sets (or other category) satisfying gluing conditions. See Section 5.3.

String Diagram A graphical calculus for monoidal categories where objects are wires and morphisms are boxes. See Section 2.4.

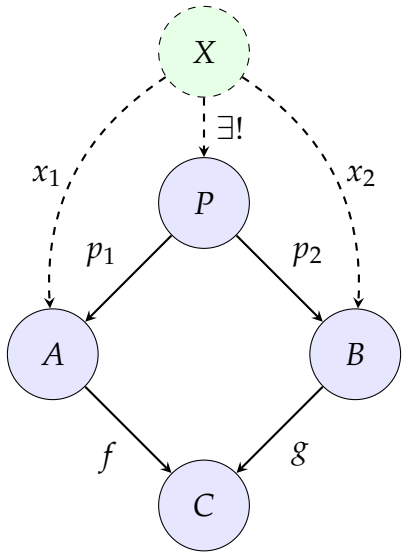
Topos A category with finite limits, exponentials, and a subobject classifier; generalises the category of sets. See Section 6.3.

Traced Monoidal Category A monoidal category with a trace operation allowing feedback loops. See Section 3.8.

Yoneda Lemma States that natural transformations from a representable functor correspond to elements of the representing object. See Section 2.6.

C Supplementary Diagrams

This section provides additional diagrams illustrating key categorical concepts.



Pullback: universal among objects with compatible maps to A and B

Figure 43: The pullback as a limit: P is the universal object with maps to A and B that compose to equal morphisms into C.

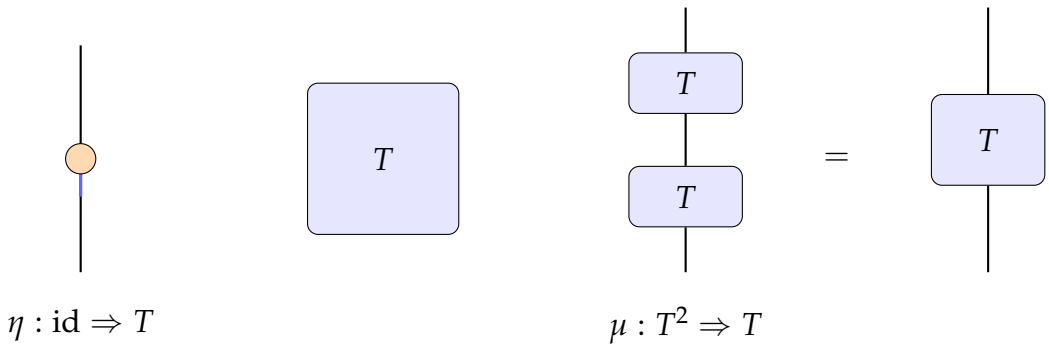


Figure 44: A monad T with unit η (introducing the monad) and multiplication μ (collapsing nested applications).

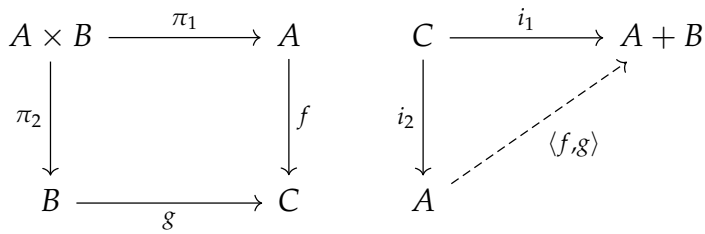


Figure 45: Product (left) and coproduct (right) as dual universal constructions.

D Further Reading

For readers wishing to deepen their understanding, we recommend the following resources organised by topic.

D.1 Category Theory Foundations

- **Introductory**: Awodey (2010) provides an accessible introduction suitable for computer scientists; Leinster (2014) is concise and example-rich.
- **Comprehensive**: Mac Lane (2013) is the classic graduate text; Riehl (2017) offers a modern treatment emphasising categorical thinking.
- **Applied**: Fong and Spivak (2019) connects category theory to diverse applications; Barr and Wells (1990) targets computer science.

D.2 Categorical Logic and Type Theory

- Jacobs (1999) provides a comprehensive treatment of categorical logic;
- Pitts (2000) surveys categorical semantics for type theory;
- Wadler (2015) gives an accessible introduction to the Curry-Howard correspondence.

D.3 Monoidal Categories and String Diagrams

- Selinger (2010) surveys graphical languages for monoidal categories;
- Coecke and Kissinger (2017) develops string diagrams for quantum computing, with transferable insights.

D.4 Categorical Probability

- Fritz (2020) develops synthetic probability in Markov categories;
- Heunen et al. (2017) provides categorical semantics for probabilistic programming.

D.5 Deep Learning Foundations

- Goodfellow et al. (2016) is the standard textbook for deep learning;
- Bronstein et al. (2021) covers geometric deep learning comprehensively;
- Bengio et al. (2013) surveys representation learning.

D.6 Categorical Deep Learning

- Fong et al. (2019) establishes the lens perspective on backpropagation;
- Cruttwell et al. (2022) develops the Para construction for gradient-based learning;
- Gavranović (2024) provides a comprehensive categorical treatment.